

How to create a Platform Application In Azure

Introduction

A Platform is a set of applications working together.

This document provides a complete template describing all steps required to create a Platform Application in Azure.

The goal is to run multiple applications (each encapsulated in its own container) together with back-end resources in parallel. These applications must only exchange information when an internal credential or identity (an Azure User Assigned Managed Identity) is provided.

The entry point of the platform is a public API. External clients — such as Angular applications, mobile applications, or testing tools like Postman — can connect to this API only by presenting a valid token issued by the Platform.

To be precise, the platform contains the following components:

- SampleApi — the main API exposed to external clients.
- SampleDataAccessApi — the internal API responsible for database access
- Azure SQL Database — the persistent data store

The platform has been developed using .NET Core (.NET 8.0) and Entity Framework Core as the ORM (Object-Relational Mapping).

As mentioned earlier, the platform implements two security layers:

- Service-to-Service Authentication — implemented with an Azure User Assigned Managed Identity
- User Authentication — where users must present a valid token to access the application (for example, to add or delete records). Here JWT, Microsoft Entra ID, and Role-Based Authorization play an important role.

This document illustrates how to containerize this platform in Azure and how Azure DevOps can be used to host the code and test the platform using Postman.

Azure Concepts

The first thing we need to do is to understand or to clarify some important concepts in Azure:

- **Resource:** In Azure when create objects/instances it will also create a resource for this instance. A resource stores metadata about the instance: resource ID, Object ID, Resource Group, Region, RBAC rules (who can use based on roles), Tags, Policies (for compliance), Life-cycle (when will be created, deleted or moved), Monitoring which that the resource can

generate Logs, Metrics and alerts. This metadata is necessary to manage the object inside the Azure Universe: where is the object, who has access etc

And also Azure will allow us to group similar resources in groups: resource groups. An analogy would be: the instance=letter, the resource=envelop, resource group=folder.

Some Azure objects/instances can be shared between applications, but each of them always has its own Resource (envelope). Shared objects correspond to infrastructure components such as Logs (Log Analytics Workspace), User Assigned Identity, Key Vault, Storage Account, or Virtual Network. Objects that contain code or data must always have their own dedicated Resource because they need to be isolated: Container App, SQL Database, App Service. The reason why some objects/instances are shared is cost, central management, and security — but the underlying Resource is never shared.

In order to group resources in resource groups the rule would be:

Same Life-cycle: when resources are deployed, deleted, versioned, migrated together.

Same Responsibility: they are developed, deployed by the same team and they use common pipelines.

Same Security Limits: apply to them the same RBAC rules, policies and compliance.

In case that all these rules apply then means that we can put all the resources that have all this in common in the same resource group.

For our project, the SampleApiPlatform, where we can have several containers as well as Databases we have the following Resource Groups:

For our global project the SampleApiPlatform where we have databases and different containers running in parallel we will have the following resource groups:

Platform Resource Group Structure (Optimal)

- **sampleApiPlatform_rg** The central resource group (RG) for platform-wide infrastructure. Everything that is shared, long-lived, security-critical, or platform-wide belongs here: **User Assigned Identities, Key Vault, Log Analytics Workspace, Virtual Network, Storage Accounts (if used platform-wide), Private DNS Zones, Shared Policies / RBAC**

This is the heart of your platform.

Everything that multiple apps use belongs here.

- **sampleApiApps_rg** The resource group for all container apps, meaning the actual microservices: **SampleDataAccessApi (Container App), SampleMarketingApi, (Container App), SampleSalesApi (Container App), Frontend apps, Sidecar services, Revision management / deployments**

Apps have a different lifecycle than infrastructure.

Apps are redeployed frequently; identities are not.

- **sampleApiData_rg** The resource group for data, meaning everything persistent: **Azure SQL Database, Azure SQL Server / Elastic Pool, Private Endpoint for SQL, Auditing / Threat Detection, Backup policies**

Data has the strictest security boundary.

Data outlives apps and outlives identities.

- **sampleApiNetwork_rg** (optional) If you want to keep networking separate (enterprise standard): Vnet, Subnets, Private Endpoints, Routing / Firewall / NSGs
Optional for small projects, standard for large platforms.

Why this structure is perfect?

- **Lifecycle:** Platform resources live a long time, Apps are constantly redeployed, Data lives the longest. Conclusion: Three distinct lifecycles mean three resource groups.
- **Responsibility:** The platform team manages identities, Key Vault, and logs. The app team manages container apps. The data team manages SQL. Conclusion: Three responsibilities mean three resource groups.
- **Security Limits:** The platform has shared RBAC. Apps have app-specific RBAC. SQL has extremely strict RBAC. Conclusion: Three security boundaries mean three resource groups.

- **Container:** packed application that contains everything to run: code, runtime (.net core), libraries, Configuration

Containers are isolated, portable, reproducible, start fast, and they do not have conflicts with other applications.

- **Container Image or Image:** instructions to build a container. They are specified in Docker file, stored in a registry container like ACR, Dockerhub. The Azure Container Apps pulls the image from the registry and starts it.
- **Container App:** is a container that is hosted and run in Azure, with the following extras: autoscaling, Ingress (HTTP / HTTPS), Secrets, Revisions, Logs, Managed Identity.

Ingress: when the container app will provide services to other users through the internet then we need to activate Ingress. Ingress provides the app an url, routing by redirecting the app to the right Revision, Load balancing between different container instances. Ingress also manages certificates such as: TLS/HTTPS as well as port mapping.

Revision = Snapshot of the Image and properties of the container app. It is a version of the container app. A new version is created automatically by Azure when the image, the environment variables, the port, a secret, or an ingress property has changed. The revisions are very useful for the following scenarios:

Rollback — You can revert to an older revision at any time.

Traffic Splitting — You can route 10% of traffic to the new revision and 90% to the old one.

Blue/Green Deployments — Test the new version before you push it live.

Stable Releases — The old revision remains active until the new one is running stably.

Zero Downtime — The new revision starts up while the old one continues to run.

Debugging — You can view logs per revision.

A Container App has only one container PER REVISION. If we have 3 revisions we have 3 containers: Revision A (1 container), Revision B(1 Container), Revision C(1 Container). In other environments instead of Container Apps they use Pods where the containers are hosted and each Pod can contain several containers.

Replica: it is an instance of a container and they exist to manage cases of elastic demand. By default we have 1 Revision with one container running, i.e 1 Replica. If suddenly the demand grows, another instance of the container (replica) is created. It also can happen that our replica for any reason falls over. Then another replica will be created automatically for us. Azure will create a maximum of Replicas that we define in the maxReplicas property and Azure decides when to create a new Replica dynamically depending on factors such as: $\text{cpu} > 70\%$, $\text{httpRequests} > 100/\text{s}$, $\text{queueLength} > 50$. On the otherhand it can reduce the amount of replicas automatically for us when for instance $\text{cpu} < 20\%$.

Managed Identity: Azure gives to our Container App an account, an Identity for our Container App and there is no need of Password, Secrets, Certification, Key Vault etc. because everything is managed by Azure itself. This is very useful when our application needs to connect with other Azure services such as Azure SQL, Azure Key Vault, Azure Storage, Azure Container Registry (ACR), Service Bus, Event Hubs, etc.

Then we can assign a role to our identity to limit the scope of what can do or Role Based Identity (RBAC). For instance these roles:

AcrPull: container app can pull images from ACR

Key Vault Reader: the container App can read secrets

Storage Blob Data Container: Container App can write Blobs

We can configure an API at Enterprise Level Security, meaning that it does not need any connection strings, any password, and that Login is managed through Managed Identity.

However Managed Identity cannot be used in case that this application is called from outside Azure: Postman, local application from our laptop, mobile, an AWS application, etc.

The Managed Identity is not automatically activated for Container Apps and has to be explicitly activated. To activate the managed identity from the portal, go to the container > Security > Identity > System Assigned > Set the status = on.

Other authentication methods can be: User defined authentication (go to the container > Security > Identity > User Assigned) or use an identity provider such as Microsoft, Apple, Facebook, Google, Twitter, Open ID connect (go to the container > Security >

Authentication). The identity provider allow us to login into another application using Tokens.

The main difference between User Assigned Identity and Managed Identity:

System Assigned (Managed Identity)	User Defined Identity
• Azure automatically creates ONE identity for your Container App as soon as you enable it. • Belongs to this Container App only. • Is automatically deleted when the app is deleted. • You can only have one System Assigned Identity per Container App. • Ideal for ACR Pull, SQL Login, Key Vault, and Storage. • Simplest approach → covers 95% of use cases	• You create the identity yourself as a separate Azure object. • Can be used by multiple Container Apps • Persists even when the Container App is deleted • You can assign multiple User Assigned Identities to a single app • Ideal for microservices that need to share the same identity • Ideal for shared access (e.g., multiple apps accessing the same Key Vault)

Just to clarify, User Defined or Managed Identities are assigned to applications (container apps) but not to resources (Azure SQL).

Port Mapping: a container App has ONE EXTERNAL port to the outside world, ex 443 (HTTPS), 8080 (HTTP), to where the all containers in each revision listens to (common port), and is used by all revisions. Strictly speaking these external port does not belong to the App Container but to the Ingress Gateway At the same time each container in each revision has an INTERNAL port (ex. 8080) but all share this port number. So all containers in each revision will have the same port number in this case 8080. The port Mapping in the Container App maps the external port to the internal container port: 443 > 8080.

- **Container Apps Environment:** it is the platform where our container app is running. It contains: internal network, ingress controller, secret storage, revision system, log streaming, compute cluster (something similar to Kubernetes), Zone Redundancy. In summary all the infrastructure that makes possible each revision to run.
- **Availability Zone:** a group of Data centers (minimum 3). An availability zone will have its own electricity supply, cooling system, network, hardware. They are separate from each other but logically connected.

In West Europe for instance there are 3 Availability zones.

- **Zone Redundancy:** this concept is used at several levels in Azure. Essentially means that when enabled, our App Container will run in different Availability Zones in the same region.

When disabled the App Container runs only in one Availability Zone.

These Availability Zones, are near enough to communicate fast but far enough to avoid the same accident: fire, electricity cut off, hardware-crash.

When there is an issue in one Availability Zone then the App Container can run from another Availability Zones (back up) and therefore there will not be any down-times. When the zone redundancy is disabled this will not be possible, however, if the issue is local in one data center in the availability zone and other data centers in the same availability zone are not

affected, then the application can continue running in another data center in the same availability zone.

The SampleDataAccessApi Application

Create an image in Dockerhub

The first step before to containerize an application is to have an image repository. For this we have several options: Azure ACR, Github, etc. In this document we have chosen **Dockerhub** mainly because is free and with Github we have encountered some technical issues.

We will need the first time the image into docker, otherwise App Container is unable to create a container for our App as this data is required at creation time. To avoid creating a local image in an environment that is not the same as in azure, we will create an **azure-pipeline-first-image.yml** that will create an image in dockerhub.

trigger:

- main

pool:

- vmImage: 'ubuntu-latest'

steps:

- checkout: self

- script: |

- export DOCKER_BUILDKIT=1

- displayName: Enable BuildKit

- task: DockerInstaller@0

- displayName: Install Docker

- script: |

- docker build -t popdemar/sampledataaccessapi:latest .

- displayName: Build Docker Image

- script: |

- echo \$(DOCKERHUB_TOKEN) | docker login -u popdemar --password-stdin

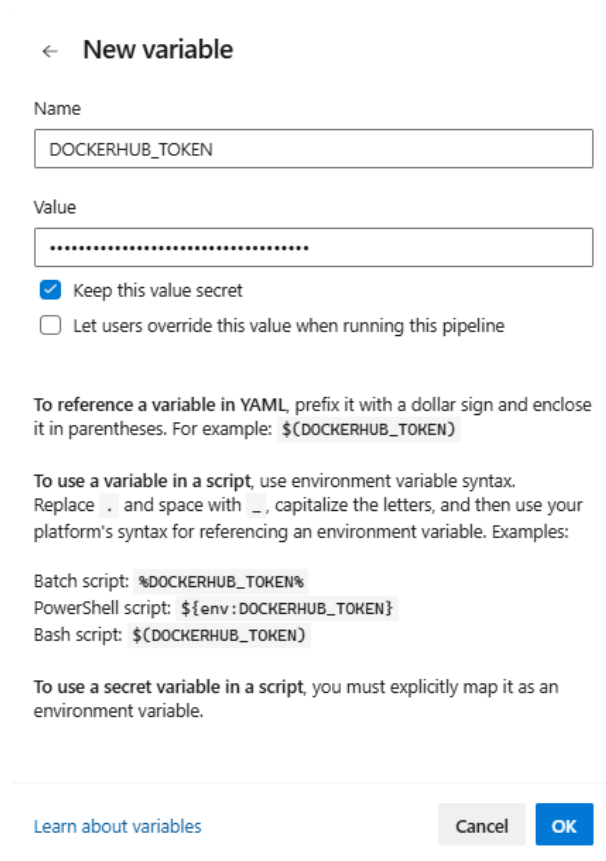
- displayName: Docker Login

- script: |

- docker push popdemar/sampledataaccessapi:latest

- displayName: Push Image to Docker Hub

In order to run this script we will need to create an environment variable called **DOCKERHUB_TOKEN** that stores the password to our Docker account (it is not necessary to create a Personal Access Token or PAT for this case). Make sure to make to check the **Keep this Value Secret** checkbox.



← New variable

Name

DOCKERHUB_TOKEN

Value

.....

☒ Keep this value secret

☐ Let users override this value when running this pipeline

To reference a variable in YAML, prefix it with a dollar sign and enclose it in parentheses. For example: `$(DOCKERHUB_TOKEN)`

To use a variable in a script, use environment variable syntax. Replace `.` and space with `_`, capitalize the letters, and then use your platform's syntax for referencing an environment variable. Examples:

Batch script: `%DOCKERHUB_TOKEN%`

PowerShell script: `${env:DOCKERHUB_TOKEN}`

Bash script: `$(DOCKERHUB_TOKEN)`

To use a secret variable in a script, you must explicitly map it as an environment variable.

[Learn about variables](#) Cancel OK

Figure 1: screenshot Screenshot environment variable DOCKERHUB_TOKEN

And this is the **Dockerfile** that we are using (make sure that has been pushed in the repo with the rest of the code):

```
# Build stage
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
ARG BUILD_CONFIGURATION=Release

WORKDIR /src

# Copy csproj
COPY SampleDataAccessApi/SampleDataAccessApi.csproj SampleDataAccessApi/

# Restore
RUN dotnet restore "SampleDataAccessApi/SampleDataAccessApi.csproj"

# Copy the rest of the source code
COPY SampleDataAccessApi/ SampleDataAccessApi/
```

```
# Build
RUN dotnet build "SampleDataAccessApi/SampleDataAccessApi.csproj" -c
$BUILD_CONFIGURATION -o /app/build

# Publish
RUN dotnet publish "SampleDataAccessApi/SampleDataAccessApi.csproj" -c
$BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false

# Runtime stage
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app

COPY --from=build /app/publish .

ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080

ENTRYPOINT ["dotnet", "SampleDataAccessApi.dll"]
```

The **image and tag** for simplicity we chose the **latest**. It is possible to indicate a specific build.

Once we complete this stage we will have our image, sampleaccessapi, uploaded in the Dockerhub and now we will be able to create a container in Azure.

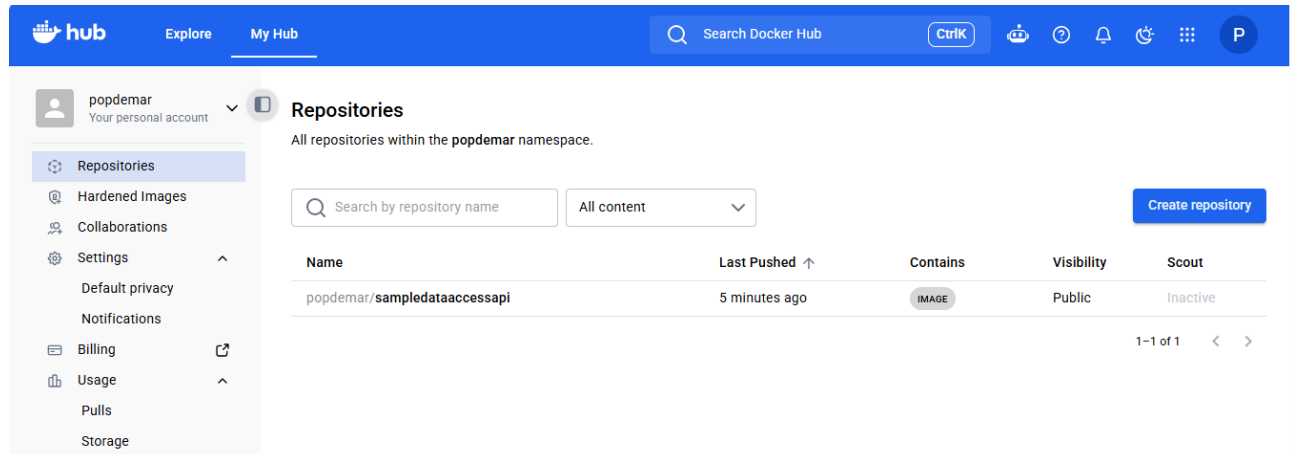
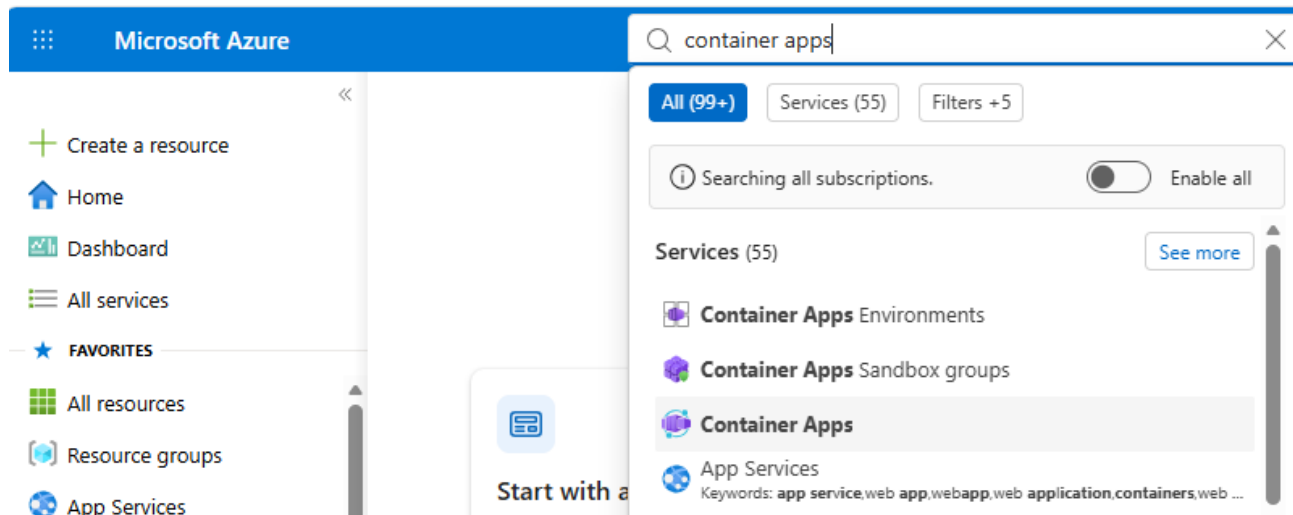


Figure 2: screenshot DockerHub image created

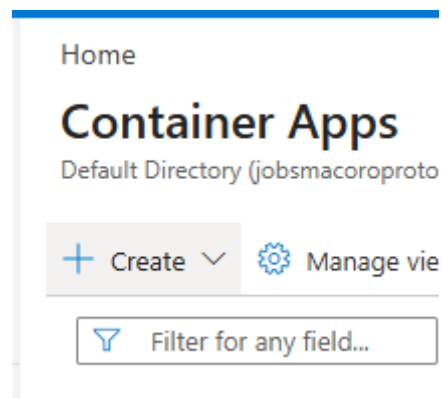
Create Container App in Azure

Before deploying and running our pipeline we need to create in Azure a container in the Container App section as follows:

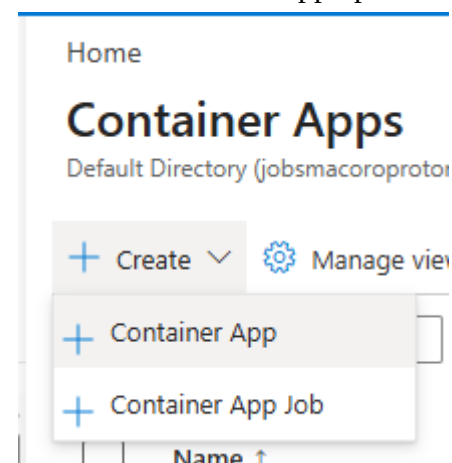
Navigate to Container Apps:



Click on Create:



Select the Container App option:



Fill up the form with the following values:

Azure Subscription = your subscription

Resource Group = SampleDataAccessApi_rg (create a new resource group)

Container app name = sampledataaccessapi

Optimize For Azure Functions = False (Unchecked)

Deployment Source = Container Image

Container Apps Environment:

Show Environment In All Regions = False (Unchecked)

Region= France Central (Your Region)

Container Apps Environment= sampledataaccessapi-env (create a new one)

Create Container App:

Container Details

Name= sampledataaccessapi

Image Source = Dockerhub or other registries

Image type = public

Registry login server = docker.io

Image and tag = popdemar/sampledataaccessapi:latest

Command override = empty

Arguments override = empty

Development Stack Specific Features:

Development stack = Unspecified

Container Resource Allocation:

Workload Profile=Consumption-Up to 4 vCPU 8GB memory

GPU = False (Unchecked)

CPU and Memory = 0.5 CPU Cores , 1 Gi memory

Environment Variables:

ConnectionStrings__MoviesDb=secretRefMoviesDb

Create Container app ...

Select a subscription to manage resource creation and costs, and a resource group to organize all your resources for this deployment.

Subscription *	Azure subscription 1
Resource group *	(new) sampleDataAccessApi_rg Create new resource group
Container app name *	sampladataaccessapi
Optimize for Azure Functions	☐ ☒ Built-in support and autoscaling for Azure Functions (requires image compatible with Functions). How to run functions with your container app
Deployment source *	☒ Container image Bring your own container registry or build a container from a Dockerfile. ☐ Source code or artifact Build and deploy your code without using a Dockerfile.
Container Apps environment An environment is a secure boundary around a group of container apps. Container Apps Pricing	
Show environments in all regions ⓘ	☐
Region *	France Central
Container Apps environment *	(new) sampladataaccessapi-env (sampleDataAccessApi_rg) Create new environment

Figure 3: screenshot Create Container App

Notice that we have created:

- new resource = SampleDataAccessApi_rg
- new environment = sampladataaccessapi-env

For this purpose fill out the form with the following values:

Environment Name= sampladataaccessapi-env

Zone Redundancy=disabled

Create Container Apps environment ...

i GPU enabled workload profiles are now available. Please note GPU enabled workload profiles can only be added at the time of Azure Container Apps Environment creation.

Environment details

The environment is a secure boundary around one or more container apps that can communicate with each other and share a virtual network, logging, and Dapr configuration. [Learn more](#)

Environment name *

sampledataaccessapi-env ✓

Zone redundancy

A Container App Environment can be deployed as a zone redundant service in the regions that support it. This is a deployment time only decision. You can't make Container App Environment zone redundant after it has been deployed. [Learn more](#)

Zone redundancy *

- ☒ **Disabled:** Your Container App Environment and the apps in it will not be zone redundant.
- ☐ **Enabled:** Your Container App Environment and the apps in it will be zone redundant. This requires vNet integration.

Create

Cancel

Figure 4: screenshot Createtion of the Container Apps Environment

Next we have to specify from which image and from which repository this container will be created.

Create Container app ...

Container details

Name *	sampledataaccessapi
Image source	☐ Azure Container Registry ☒ Docker Hub or other registries
Image type	☒ Public ☐ Private
Registry login server * ⓘ	docker.io
Image and tag *	popdemar/sampledataaccessapi:latest
Command override ⓘ	Example: /bin/bash
Arguments override ⓘ	Example: -c, while true; do echo hello; sleep 10; done

Figure 5: screenshot Create Container App

In our case we are storing our image in Dockerhub and our repository is marked in Docker as public.

In a free account Dockerhub provides 1 only repository that can be Private.

As we probably will need more cases like this for the Sample Api Platform, one repository will not be enough, and therefore we opt for a Public.

The Privacy properties in our Docker for our account can be set on the following link:

<https://hub.docker.com/repository-settings/default-privacy>

And then go to : Settings > Default Privacy..

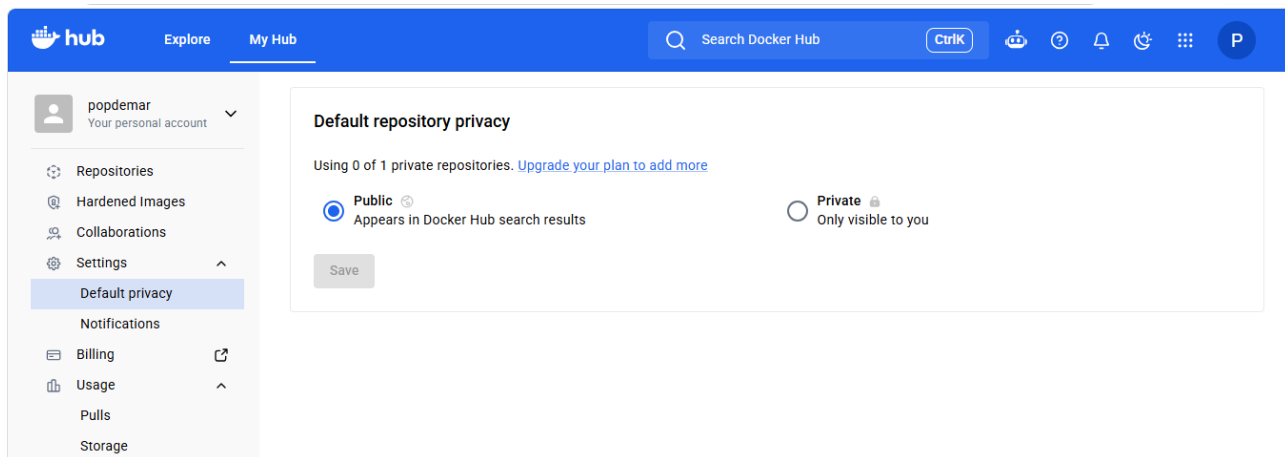


Figure 6: screenshot Dockerhub Repository Privacy = Public

Re Development stack-specific features

Al When you select a specific development stack, you get additional features tailored to that stack—optimizing Container Apps to perform for your unique settings.

Development stack Unspecified

Re Container resource allocation

Re Choose the workload profile for this app. You can adjust the CPU and memory allocation for this app up to the workload profile limit. [Learn more](#)

Workload profile * Consumption - Up to 4 vCPUs, 8 Gib memory

Im GPU ☐

Cc CPU and memory * 0.5 CPU cores, 1 Gi memory

Ar Environment variables

Name	Value	Delete
ConnectionStrings_MoviesDb	secretrefMoviesDb	
Enter name	Enter value	

[Review + create](#) [< Previous](#) [Next : Ingress >](#)

Figure 7: screenshot Container Resource Allocation Properties

The container resource allocation can be left as the default values provided.

Notice a that we provide the name of the connection string to the database as environment variable. For this we have 2 options:

1. Paste the connection string with the password: “
Server=tcp:sampleapiserver.database.windows.net,1433;Initial Catalog=free-sql-

```
db-5542591;Persist Security Info=False;User  
ID=sampleapiserveradmin;Password={your_password};MultipleActiveResultSets=False;En  
ncrypt=True;TrustServerCertificate=False;Connection Timeout=30;”.
```

This is insecure because the password is visible to all who can access this form.

2. To keep it as a secret: for this we need to coordinate this secret with azure devops. So here we only set a reference to the secret:

```
ConnectionStrings__MoviesDb = secretref:MoviesDb
```

Then in the Azure DevOps go to: Pipeline > Libraries > Variable Group > We create a new variable Group and we will add a new variable called sampleapiDBConnectionString and the value will be our connectionstring

```
( Server=tcp:sampleapiserver.database.windows.net,1433;Initial Catalog=free-sql-  
db-5542591;Persist Security Info=False;User
```

```
ID=sampleapiserveradmin;Password={your_password};MultipleActiveResultSets=False;En  
ncrypt=True;TrustServerCertificate=False;Connection Timeout=30;) and we will mark it as  
secret.
```

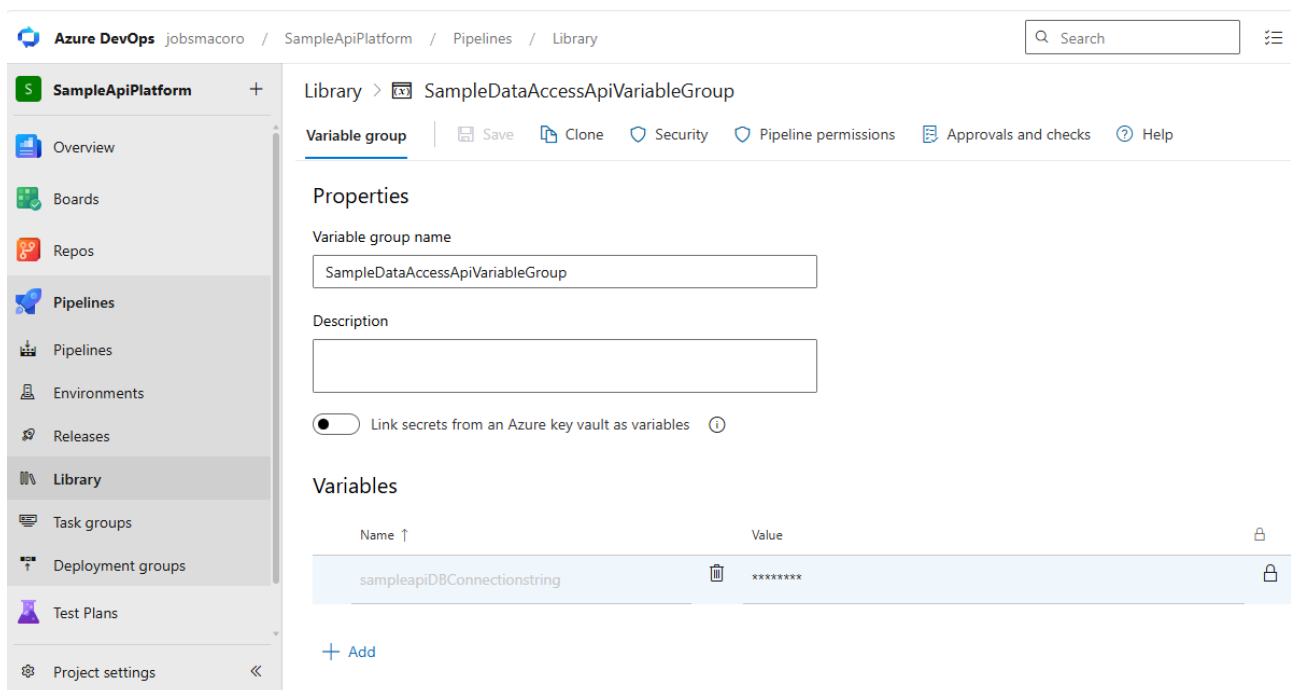


Figure 8: screenshot Azure DevOps Environment Variable

And then in the pipeline we inject this secret. We see in the pipeline a reference to this sampleapiDBConnectionString:

Make sure also that in Program.cs we reference the right connection string:

```

// EF Core SQL Azure with retry in case there are transient errors
builder.Services.AddDbContext<MoviesDbContext>(options =>
{
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("MoviesDb"),
        sqlOptions => sqlOptions.EnableRetryOnFailure(
            maxRetryCount: 5,
            maxRetryDelay: TimeSpan.FromSeconds(10),
            errorNumbersToAdd: null
        )
    );
});

```

Figure 9: screenshot Program.cs connection string

And in appsettings.json this value must be empty because they are setup in Azure.

```

// staging: Azure connection strings
"ConnectionStrings": {
  "MoviesDb": "",
  // "BillingDb": "Server=tcp:...,
  // "ProductsDb": "Server=tcp:...,
  // "OrdersDb": "Server=tcp:...,
  // "CustomerDb": "Server=tcp:...,
},

```

Figure 10: screenshot appsettings.json connection string has to be empty

After that click on Review + Create > Create

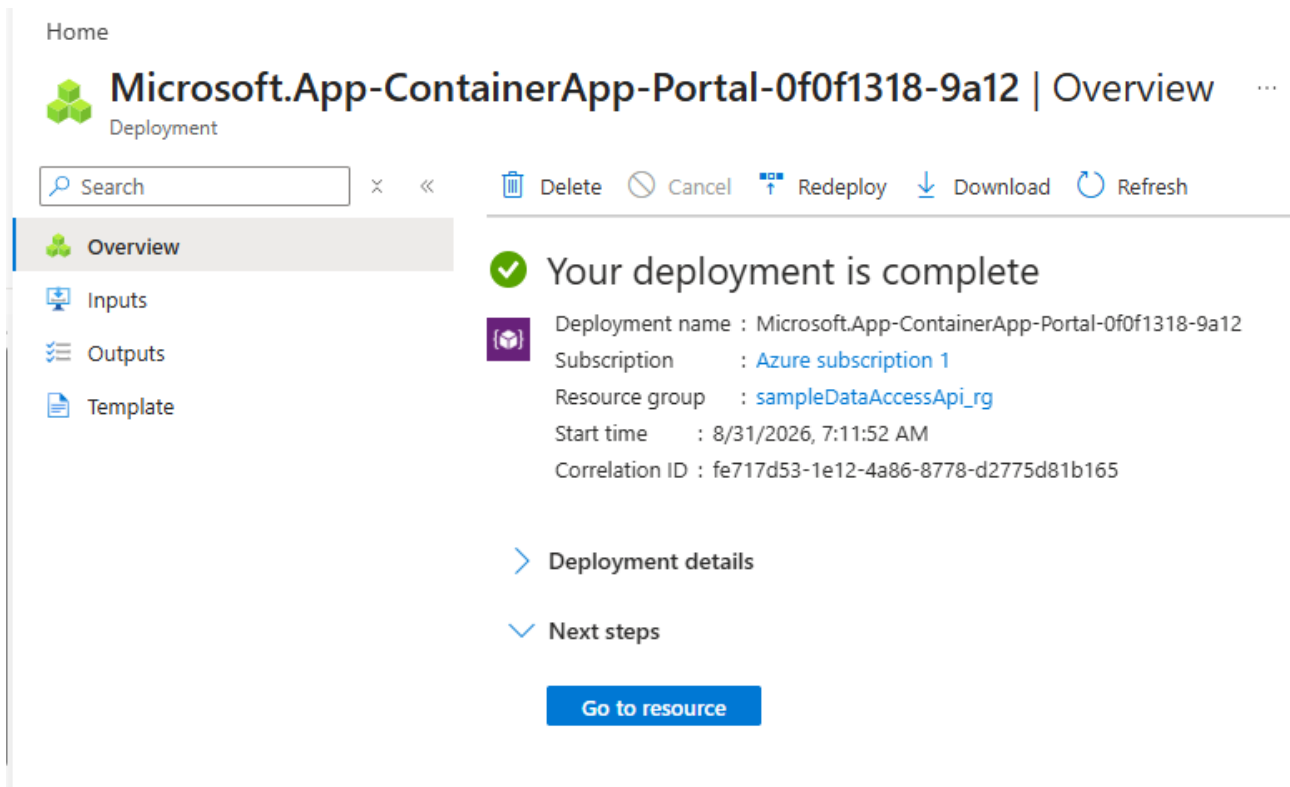


Figure 11: screenshot Container App Created in Azure

Once we receive the confirmation that Azure has created our container, we need to check if the container is running, and if there is a revision active, and if the app is reachable etc.

For this purpose in the Azure Portal go to: **Azure Portal** → **Container Apps** → **Deine App** → **Application** → **Revisions**

It should show something like this:

- **Active Revision: Yes**
- **Healthy: Yes**
- **Container: Running**
- **Image: popdemar/<repo>:latest**

When all is green then this means that the deployment was Successful.

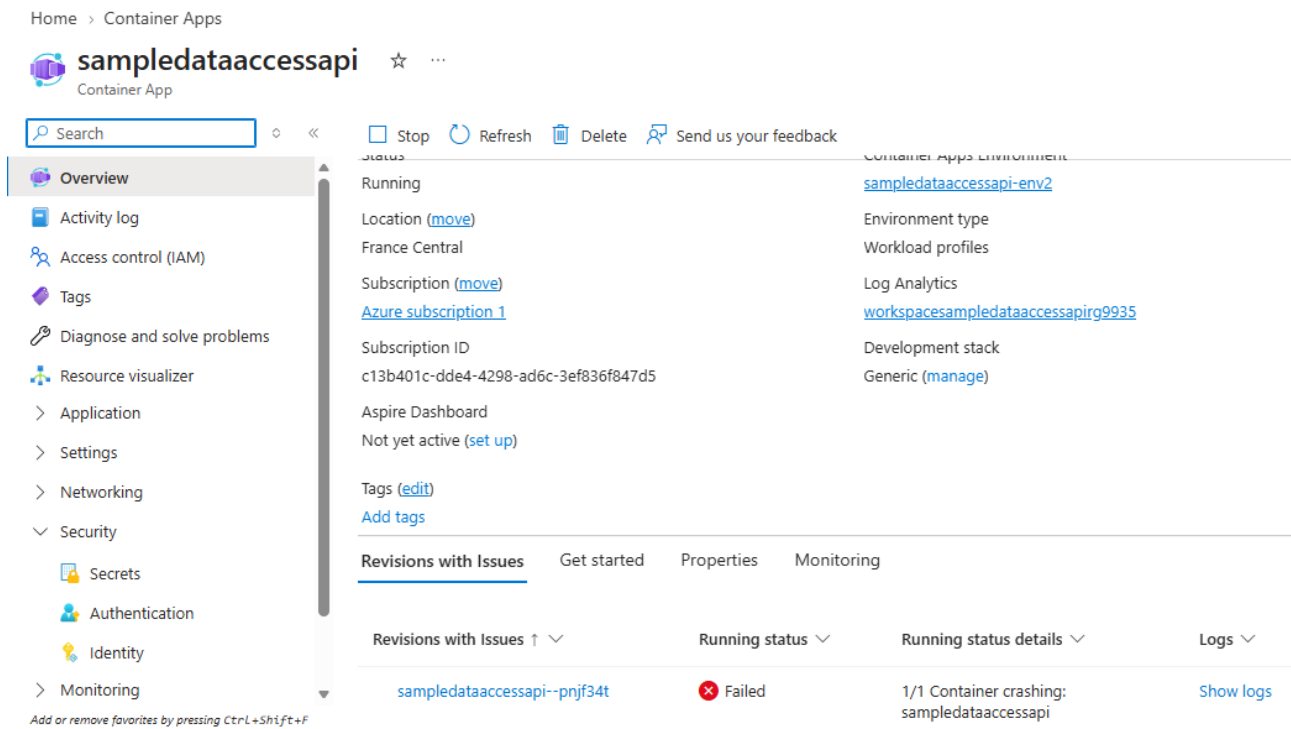


Figure 12: screenshot Container App Created but Deployment Failed

We see that our container is not running.

Troubleshooting The Container App

The quickest way to get the logs is not through the portal but through the Cloud Shell:

az containerapp logs show \

--name sampledataaccessapi \

--resource-group SampleDataAccessApi_rg \

--follow

In this case the error was that program still was reading some empty settings. We commented the code. We pushed everything to the Repo. The pipeline was triggered: a new image was deployed into the Dockerhub. However the container was still failing for the same reason because it is still using the cached image: ACA cached Images pro Revision. When a new Revision id generated the ACA always start the old Revision which still contain the old code and then it throws away the old exception.

The solution is issuing this command to force to get the last image ceated and to start a new revision.

az containerapp update \

--name sampledataaccessapi \

--resource-group SampleDataAccessApi_rg \

--image popdemar/sampledataaccessapi:latest

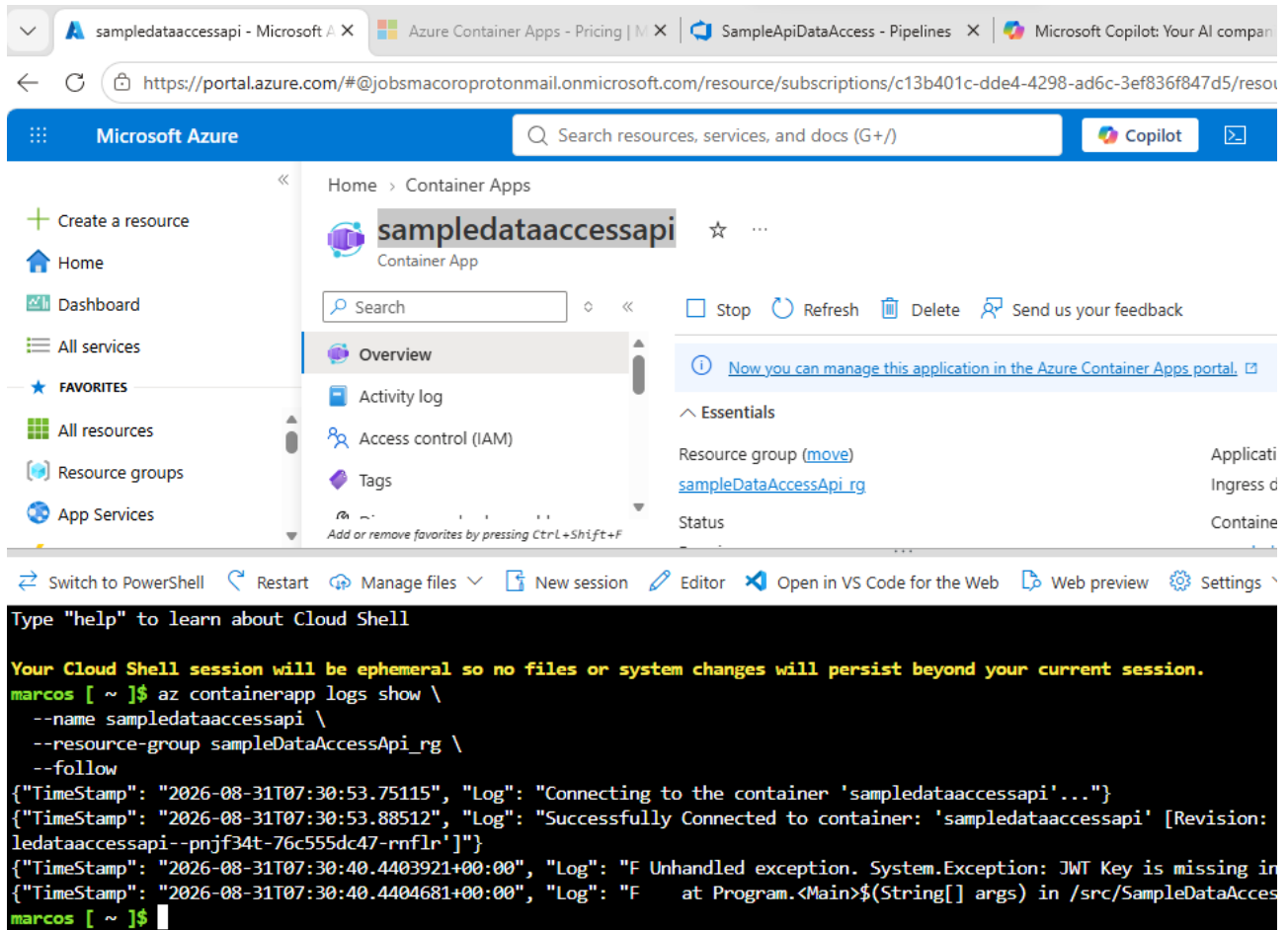


Figure 13: screenshot Azure Cloud Shell

sampledataaccessapi

Running No ingress configured

sampledataaccessapi:latest · France Central

INGRESS Ingress disabled Enable ingress

Needs attention

ScaledObjectDeleted

×11

✖ ScaledObjectDeleted: ScaledObject was removed from KEDA watch and would not be auto-scaled.

+10 more issues

Inspect logs

LIVE VITALS

● last 15 min · live

CPU

0%

0.5 vCPU allocated

Memory

0%

0 MB / 1024 MB

Requests

0/s

Avg latency: —

Replicas

0

Min: 1 / Max: 1

REVISIONS WITH ISSUES

Revision name	Status	Details	Logs
sampledataaccessapi--0000001	✖ Failed	1/1 Container crashing: sampledataaccessapi	Show logs

Figure 14: screenshot New Screen To Administer Container Apps In Azure

In this case this command did not fix the issue: we have ran the pipelines but the revision remains the same and therefore we need to kill the revision and to create a new one:

```
az containerapp revision deactivate \  
  --name sampledataaccessapi \  
  --resource-group SampleDataAccessApi_rg \  
  --revision sampledataaccessapi--0000001
```

Once the we see the "Deactivate succeeded" then issue the following command:

```
az containerapp update \  
  --name sampledataaccessapi \  
  --resource-group SampleDataAccessApi_rg \  
  --image popdemar/sampledataaccessapi:latest
```

At the end we see that the application has been successfully deployed.

What we need to add is the database connection as a secret in this container.

- Server=tcp:sampleapiserver.database.windows.net,1433;Database=free-sql-db-5542591;Authentication=Active Directory Default;Encrypt=True;
- Create a secret in the container
- Reference the secret in the environment variables of the container.

Azure Devops Pipeline Concepts

1) **Task:** a series commands that are encapsulated under the same hood. A pipeline is a list of tasks written in a yaml file. A complete list of the available tasks are available in the azure devops when we edit/create a pipeline:

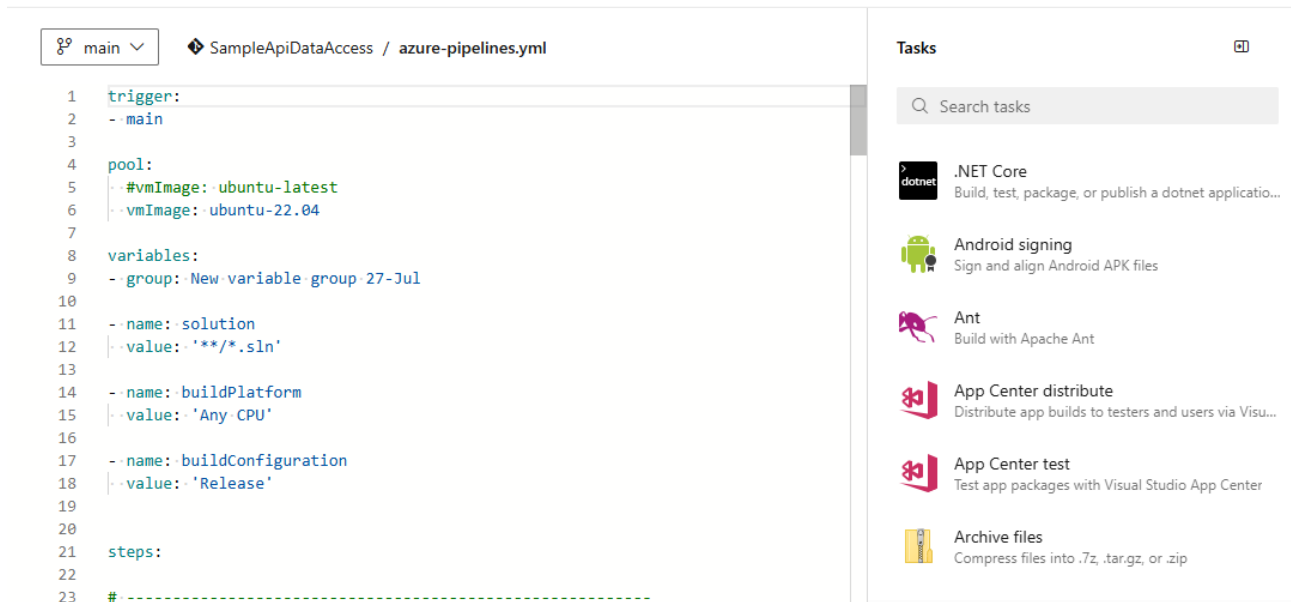


Figure 15: screenshot We can see the tasks on the right hand side

2) **Agent:** it is a machine that executes the tasks described in the pipeline on our behalf: they perform restore, builds, tests, etc on our behalf. Some agents might not have the necessary rights to execute a task (ex: to have root access to a NuGet folder). In these cases we will need to find alternative ways to execute the task.

It is also possible to give directly through commands (**script**) to the agent bypassing the tasks.

3) **Variables:** in a pipeline we can have 2 variable types: **environment variables** or **pipeline variables**. The difference is very subtle. This is important to differentiate for instance if we ask an AI to write a pipeline for us, because if we define the variables in the wrong place the pipeline will fall over.

Pipeline-Variable	<code>\$(VAR)</code>	<code>\$(DOCKERHUB_TOKEN)</code>
Bash-Environmentvariable	<code>\$VAR</code>	<code>\$DOCKERHUB_TOKEN</code>

Pipeline variables are stored in the variables section when we edit the pipelines.

The environment variables are in Pipelines > Library > Variable group.

4) **Service Connections:** it is necessary to create a service connection whenever a pipeline task or script needs to authenticate with azure or also when we are using the azure CLI. Then in the task or script we specify which Service Connection to use in order to login. For instance, and sometimes it is not obvious:

- **task:** AzureCLI@2
displayName: Update ACA image
inputs:

azureSubscription: 'SampleDataAccessApi_ServiceConnection'

scriptType: bash

scriptLocation: inlineScript

inlineScript: |

```
az containerapp update \  
  --name sampledataaccessapi \  
  --resource-group SampleDataAccessApi_rg \  
  --image docker.io/$(DOCKERHUB_USERNAME)/sampledataaccessapi:latest
```

In other occasions the Service Connection is not explicit declared in the pipeline but is implicit defined in the task and therefore we need to create a service connection for this pipeline when we use such a task. For instance to load an image from Dockerhub, we need to service connection to connect from docker to azure:

- **task:** Docker@2

displayName: Build Docker image

inputs:

command: build

Dockerfile: Dockerfile

repository: \$(DOCKERHUB_USERNAME)/sampledataaccessapi

tags: |

latest

Create a pipeline that deploys the code into the Container Application.

At this stage we have our Image in Dockerhub and an Container App to pull the image from this Image. However there are a lot of steps missing: Restore the Code, Build, Running Tests, Entity Framework Migrations. So we need to update or even to make a new pipeline that covers all these functions.

trigger:

- main

pool:

vmImage: ubuntu-latest

variables:

- name: solution

value: '**/*.sln'

- name: buildPlatform

value: 'Any CPU'

- name: buildConfiguration

value: 'Release'

#this variable is necessary to allow the agent to give access to the agent to the NuGet packages.

- name: NUGET_PACKAGES

value: \$(Agent.TempDirectory)/.nuget/packages

steps:

```
# -----
# Restore
# -----
- task: DotNetCoreCLI@2
  displayName: Restore packages
  inputs:
    command: restore
    projects: '$(solution)'

# -----
# Build
# -----
- task: DotNetCoreCLI@2
  displayName: Build solution
  inputs:
    command: build
    projects: '$(solution)'
    arguments: '--configuration $(buildConfiguration)'

# -----
# Install EF Toolsgit add
# -----
- script: |
  dotnet tool install --global dotnet-ef
  displayName: Install EF Tools

# -----
# Build EF migration bundle
# -----
- script: |
  cd SampleDataAccessApi
  dotnet ef migrations bundle --output ../efbundle
  displayName: Build EF migration bundle

# -----
# Make bundle executable
# -----
- script: |
  chmod +x efbundle
  displayName: Make EF bundle executable

# -----
# Run EF migration bundle
# -----
- script: |
  ./efbundle --connection "$(sampleapiDBConnectionString)"
  displayName: Run EF migration bundle

# -----
```

```

# Test + Coverage
# -----
- script: |
    dotnet test SampleDataAccessApi.sln --configuration Release --collect:"XPlat Code Coverage"
    displayName: Run tests with coverage

# -----
# Publish coverage
# -----
- task: PublishCodeCoverageResults@1
  displayName: Publish coverage
  inputs:
    codeCoverageTool: Cobertura
    summaryFileLocation: '$(Agent.TempDirectory)/**/*.coverage.cobertura.xml'
    reportDirectory: '$(Agent.TempDirectory)'

# -----
# Build Docker image
# -----
- task: Docker@2
  displayName: Build Docker image
  inputs:
    command: build
    Dockerfile: Dockerfile
    repository: $(DOCKERHUB_USERNAME)/sampledataaccessapi
    tags: |
      latest

- script: |
    echo "$(DOCKERHUB_TOKEN)" | docker login -u "$(DOCKERHUB_USERNAME)" --
password-stdin
    displayName: Docker Hub Login

# -----
# Push Docker image
# -----
- script: |
    docker push $(DOCKERHUB_USERNAME)/sampledataaccessapi:latest
    displayName: Push Docker image

# -----
# Update ACA image
# -----
- task: AzureCLI@2
  displayName: Update ACA image
  inputs:
    azureSubscription: 'SampleDataAccessApi_ServiceConnection'
    scriptType: bash
    scriptLocation: inlineScript
    inlineScript: |
      az containerapp update \
        --name sampledataaccessapi \

```

```
--resource-group SampleDataAccessApi_rg \  
--image docker.io/$(DOCKERHUB_USERNAME)/sampledataaccessapi:latest
```

With the following pipeline variables: DOCKERHUB_TOKEN, DOCKERHUB_USERNAME, sampleapiDBConnectionString.

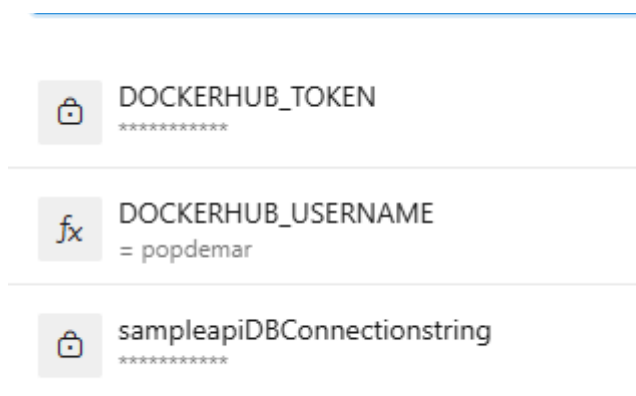


Figure 16: screenshot pipeline variables

Service Connections

New Azure service connection
Azure Resource Manager

App registration (automatic) Recommended

Need help choosing a connection type?

Credential
Workload identity federation Recommended

Scope level
☒ Subscription
☐ Management Group
☐ Machine Learning Workspace

Subscription
Azure subscription 1 (c13b401c-dde4-4298-ad6c-3ef836f8...)
If you can't find subscription you need, please [create manually](#).

Resource group
sampleDataAccessApi_rg
Specify to limit access to the chosen resource group only.

Service Connection Name
SampleDataAccessApi_ServiceConnection

Service Management Reference (optional)
Provide a Service Management Reference if your organization requires one to be configured on Entra ID App registrations.

Description (optional)

Security
☒ Grant access permission to all pipelines

[Learn more](#)
[Troubleshoot](#)

[Back](#) [Save](#)

× In this pipeline we are using Azure-CLI and therefore we need ARM-Service-Connection.

AzureDevOps > Project Settings > Pipelines > Service Connections > New > ARM (Azure Resource Manager) :

Figure 17: screenshot ARM Service Connection

We need also for this specific pipeline also a docker service connection , type: Docker registry:

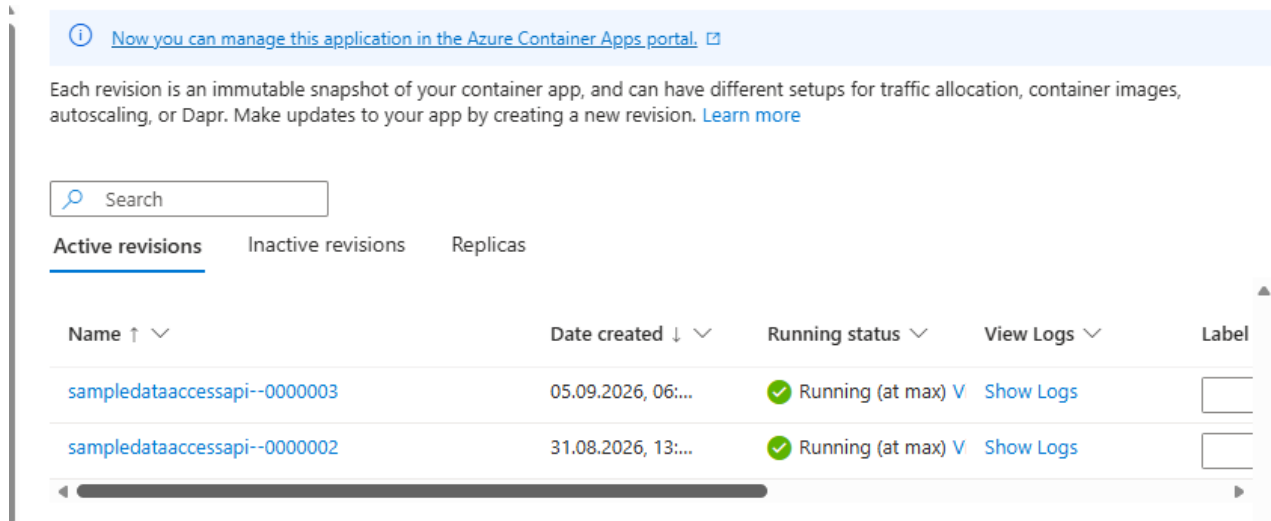


Figure 18: screenshot Docker Service Connection Type

A screenshot of the 'New Docker Registry service connection' form. The form has a title bar with a close button. Below the title, there are three radio buttons: 'Docker Hub' (selected), 'Others', and 'Azure Container Registry'. The 'Docker Registry' section contains several input fields: 'Docker Registry' (with a pre-filled URL 'https://index.docker.io/v1/'), 'Docker ID' (with 'popdemar'), 'Docker Password' (with masked characters), and 'Email (optional)'. Below these fields is a 'Verify' button and a green checkmark with the text 'Verification Succeeded'. The 'Service connection details' section includes a 'Service Connection Name' field (with 'DockerHub_ServiceConnection') and a 'Description (optional)' field. At the bottom, there is a 'Security' section with a checked checkbox 'Grant access permission to all pipelines'. The footer contains links for 'Learn more' and 'Troubleshoot', a 'Back' button, and a 'Verify and save' button with a dropdown arrow.

Figure 19: screenshot Inputs For A Docker Service Connection

After all this setup when the pipeline runs a new revision will be created for our pipeline.



The revision 03 was created.

Creating A Managed Identity

Among the Identity managed by Azure, Managed Identity and User Identity, this is the one that gives more flexibility in terms that is not a 1:1 relation between containers but a 1:n, so a we can have several containers that use the same identity to access a container.

An Identity is an object in itself in Azure. We create a User Identity:

Type in the search box of the Azure Portal: Managed Identities:

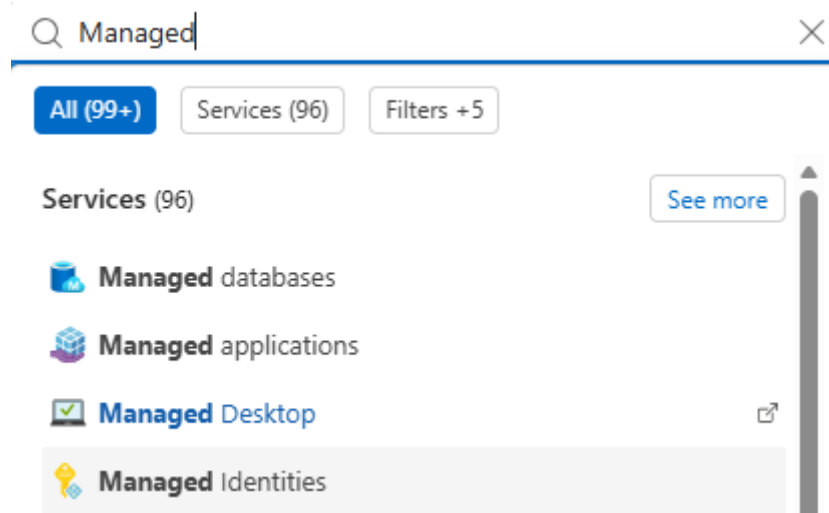


Figure 20: screenshot Managed Identities Search

An screen with the current existing identities will be presented:

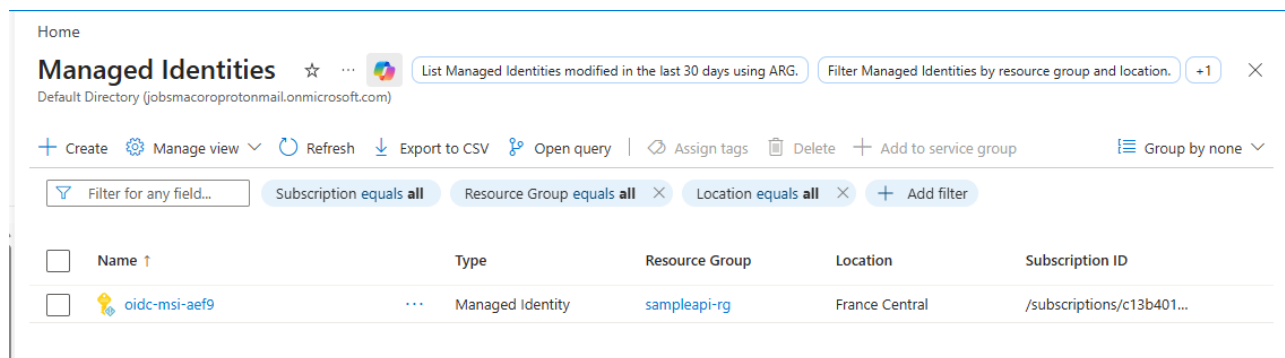


Figure 21: screenshot Current Identities List

Click on the Create button. And the following screen will be presented:

The screenshot shows the 'Create User Assigned Managed Identity' form. The breadcrumb is 'Home > Managed Identities'. The title is 'Create User Assigned Managed Identity'. There are three tabs: 'Basics' (selected), 'Tags', and 'Review + create'. Under 'Project details', there's a description: 'Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.' Below this are two dropdowns: 'Subscription' (selected: 'Azure subscription 1') and 'Resource group' (selected: '(New) sampleApiPlatform_rg'). There's a 'Create new' link below the resource group dropdown. Under 'Instance details', there are two dropdowns: 'Name' (selected: 'SampleDataAccessApi_UserKey' with a green checkmark) and 'Region' (selected: '(Europe) France Central'). Under 'Restrictions', there are two dropdowns: 'Isolation Scope' (selected: 'None') and 'Resource (Preview)' (selected: 'Microsoft.App/*' with an 'Edit' link). At the bottom, there are three buttons: 'Previous', 'Next', and 'Review + create'.

Figure 22: Create A User Defined Identity

Resource group = sampleApiPlatform_rg

Name = SampleDataAccessApi_UserKey

Region = France

Restrictions

Isolation Scope= **None** (it can be used in any region of the subscription) / Regional (can only be used by resources in the same region in the subscription). None is more flexible. If tomorrow I need to define another resource but I cannot do it in the same region then this can give some issues.

Resource (Preview) = Which resource types the identity is allowed to manage = **Microsoft.App** (we want this User Defined Identity to control the access to the container, so only containers can use this identity)

Now we go to the Container App and we will see this User Defined Identity in the list of available identities that we can use:

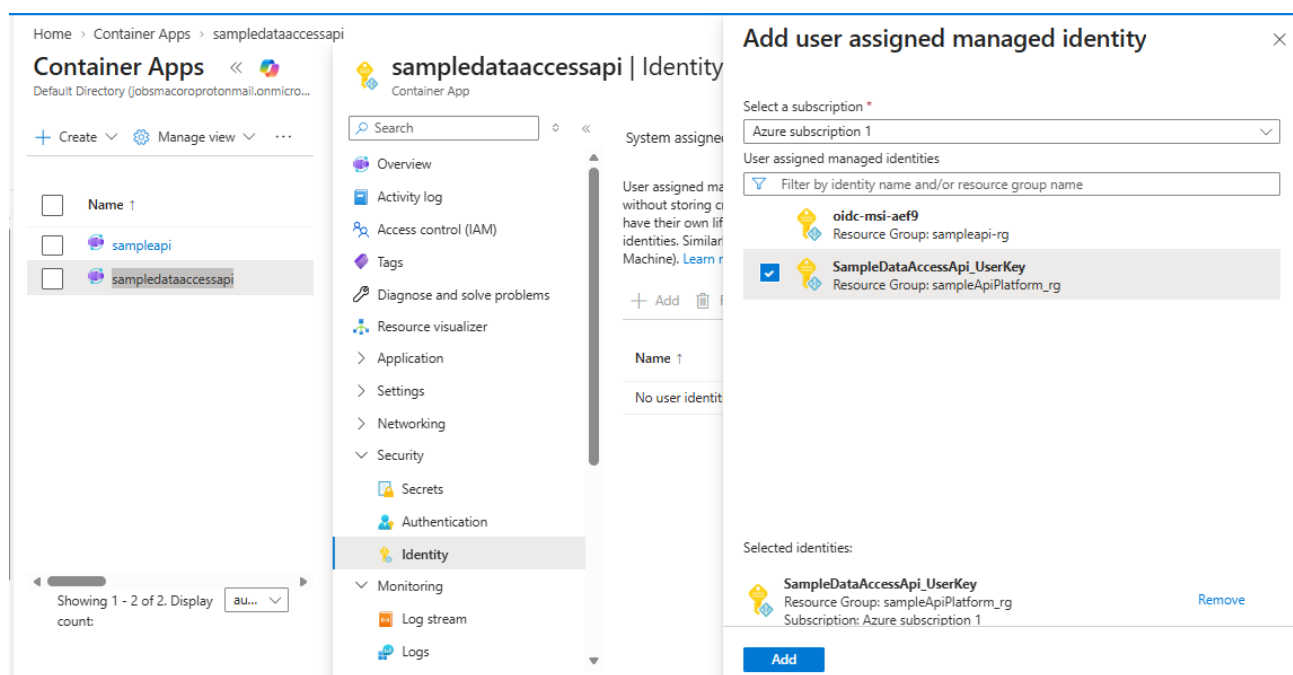


Figure 23: Adding The User Defined Identity To The Container App

After clicking on the Add button:

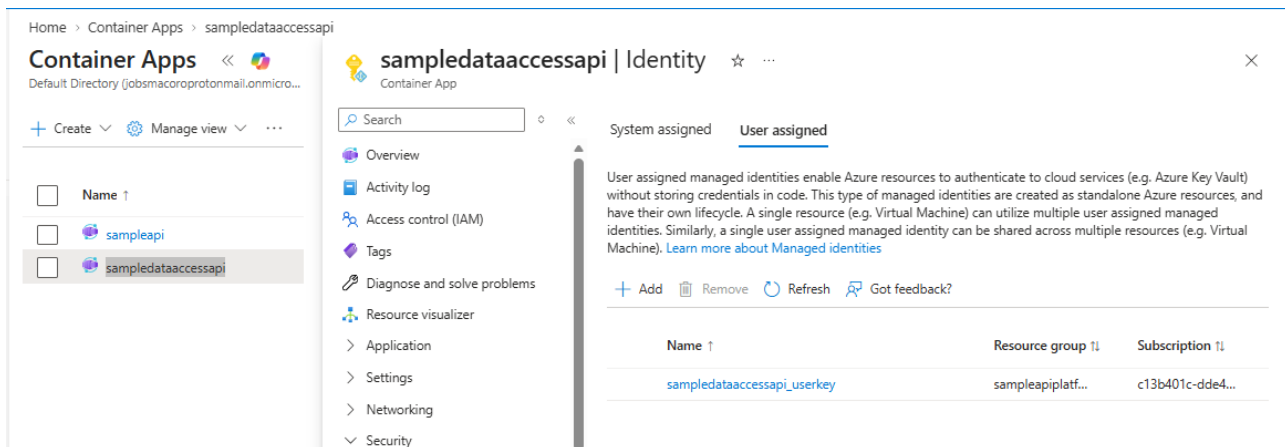
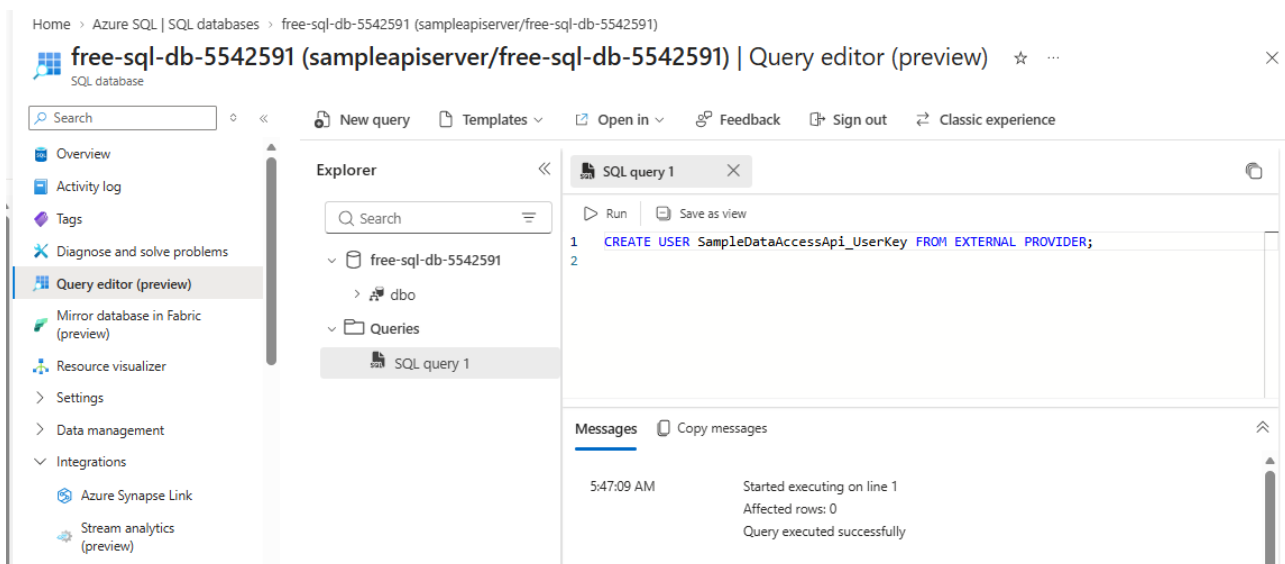


Figure 24: User Defined Identity Added To The Container App

Now we need to add permissions to this User Identity: for instance to give access to our Azure SQL Database.

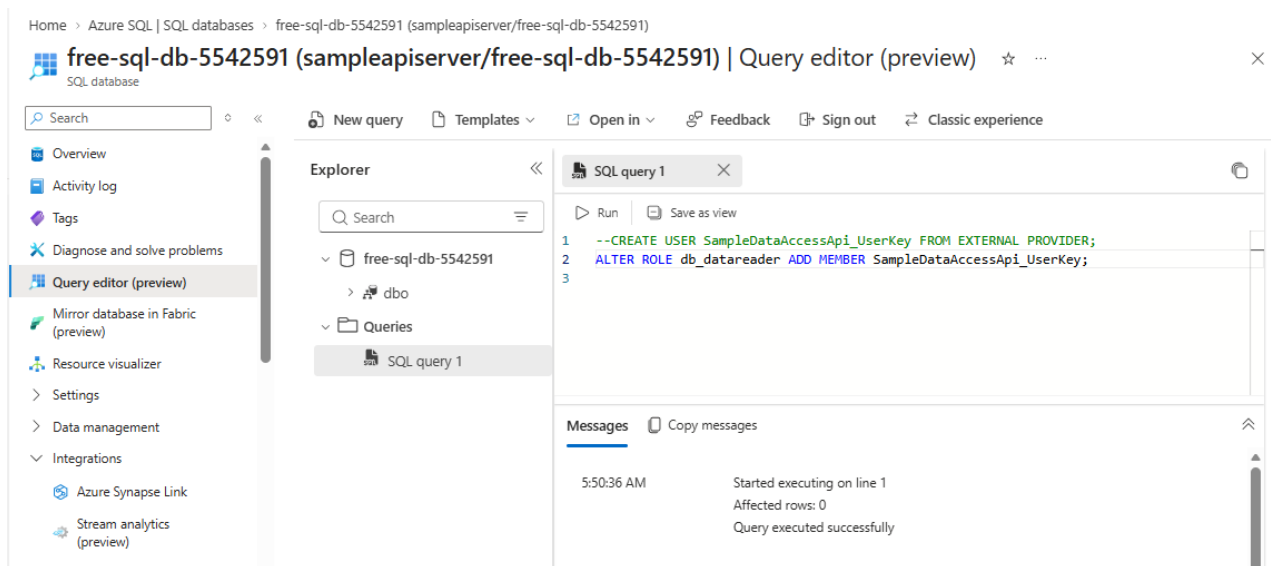
We need to login into our Azure Database with an Entra ID admin credential (in our case as jobs.macoro), not as a database admin (sampleapiserveradmin), and add our User Defined Identity, SampleDataAccessApi_UserKey, so that the database recognizes as a user and can login without a password (behind the scenes Azure will exchange a token between the database and the container).

CREATE USER SampleDataAccessApi_UserKey FROM EXTERNAL PROVIDER;

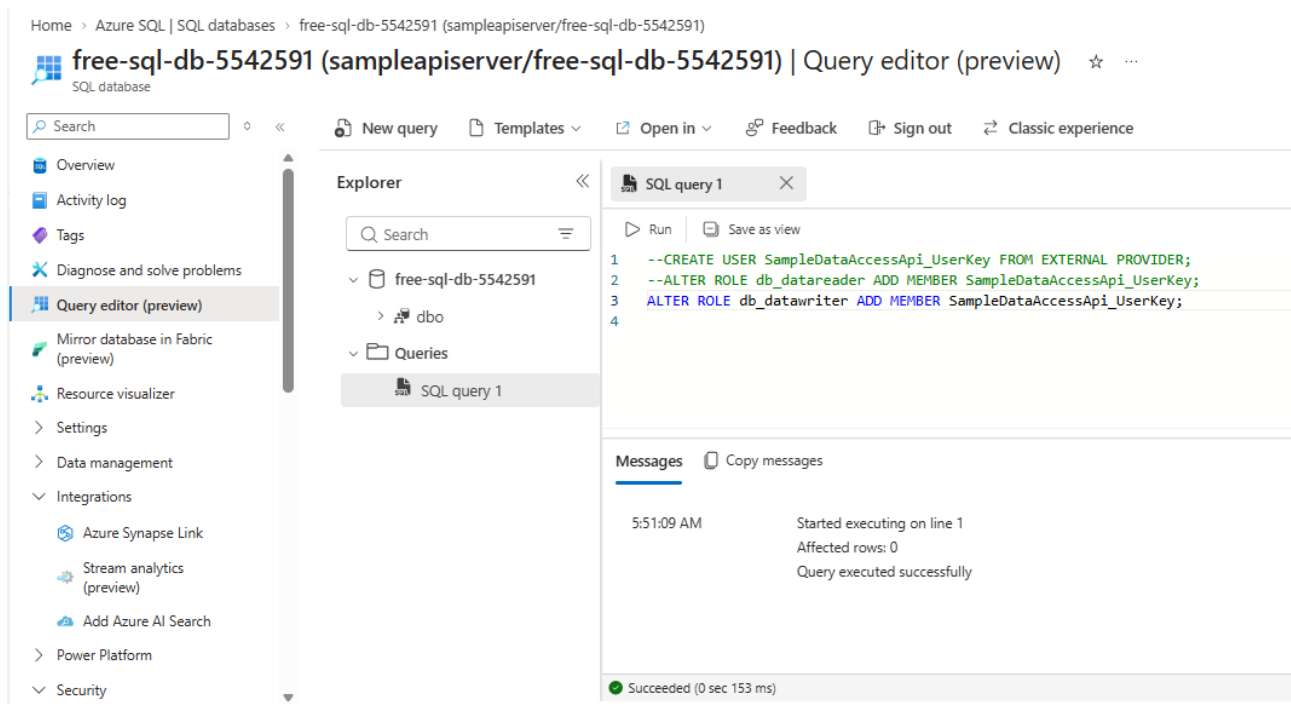


and that allows to login without password and to performed the operations.

ALTER ROLE db_datareader ADD MEMBER SampleDataAccessApi_UserKey;



ALTER ROLE db_datawriter ADD MEMBER SampleDataAccessApi_UserKey;



The SampleApi Application

Create an image in Dockerhub

azure-pipeline.yml will create an image in dockerhub.

trigger:

- main

pool:

```
vmImage: 'ubuntu-latest'
```

```
steps:
```

```
- checkout: self
```

```
- script: |  
  export DOCKER_BUILDKIT=1  
  displayName: Enable BuildKit
```

```
- task: DockerInstaller@0  
  displayName: Install Docker
```

```
- script: |  
  docker build -t popdemar/sampledataaccessapi:latest .  
  displayName: Build Docker Image
```

```
- script: |  
  echo $(DOCKERHUB_TOKEN) | docker login -u popdemar --password-stdin  
  displayName: Docker Login
```

```
- script: |  
  docker push popdemar/sampledataaccessapi:latest  
  displayName: Push Image to Docker Hub
```

And this is the **Dockerfile** that we are using (make sure that has been pushed in the repo with the rest of the code):

```
# Build stage  
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build  
ARG BUILD_CONFIGURATION=Release  
  
WORKDIR /src  
  
# Copy csproj  
COPY SampleDataAccessApi/SampleDataAccessApi.csproj SampleDataAccessApi/  
  
# Restore  
RUN dotnet restore "SampleDataAccessApi/SampleDataAccessApi.csproj"  
  
# Copy the rest of the source code  
COPY SampleDataAccessApi/ SampleDataAccessApi/  
  
# Build  
RUN dotnet build "SampleDataAccessApi/SampleDataAccessApi.csproj" -c  
$BUILD_CONFIGURATION -o /app/build
```

Publish

```
RUN dotnet publish "SampleDataAccessApi/SampleDataAccessApi.csproj" -c $BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false
```

Runtime stage

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
```

```
WORKDIR /app
```

```
COPY --from=build /app/publish .
```

```
ENV ASPNETCORE_URLS=http://+:8080
```

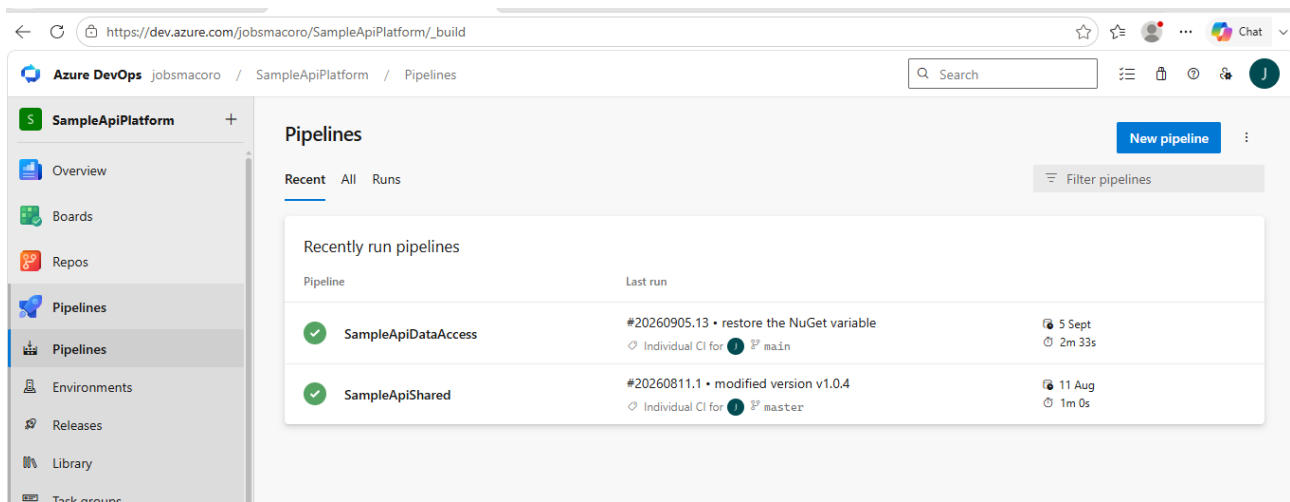
```
EXPOSE 8080
```

```
ENTRYPOINT ["dotnet", "SampleDataAccessApi.dll"]
```

The **image and tag** for simplicity we chose the **latest**.

In this case we need to create the pipeline in Azure DevOps:

Click on New Pipeline button



Then select the Azure Repos Git

Connect

Select

Configure

Review

New pipeline

Where is your code?



Azure Repos Git

YAML

Free private Git repositories, pull requests, and code search



GitHub

YAML

Home to the world's largest community of developers

More options ▾



Bitbucket Cloud

YAML

Hosted by Atlassian



Other Git

Any generic Git repository



Subversion

Centralized version control by Apache

Use the [classic editor](#) to create a pipeline without YAML.

Select the SampleAppApi Repository

✓ Connect

Select

Configure

Review

New pipeline

Select a repository

Filter by keywords

SampleApiPlatform ▾



SampleApiApp



SampleApiDataAccess



SampleApiIdentity



SampleApiShared

And then automatically it will load the pipeline that we have just created to create an image in Dockerhub.

✓ Connect

✓ Select

✓ Configure

Review

New pipeline

Review your pipeline YAML

VariablesRun

SampleApiApp / azure-pipelines.yml

Show assistant

```
1 trigger:
2   - main
3
4 pool:
5   vmImage: 'ubuntu-latest'
6
7 steps:
8   - checkout: self
9
10  - script: |
11    - export DOCKER_BUILDKIT=1
12    - displayName: Enable BuildKit
13
14  Settings
15  - task: DockerInstaller@0
16    displayName: Install Docker
17
18  - script: |
19    - docker build -t popdemar/sampledataaccessapi:latest .
```

Then we need to create the environment variable in the pipeline to store the password of Dockerhub. Click on Variables:

SampleApiPlatform / Pipelines

✓ Connect

✓ Select

✓ Configure

Review

New pipeline

Review your pipeline YAML

SampleApiApp / azure-pipelines.yml

```
1 trigger:
2   - main
3
4 pool:
5   vmImage: 'ubuntu-latest'
6
7 steps:
8   - checkout: self
9
10  - script: |
11    - export DOCKER_BUILDKIT=1
12    - displayName: Enable BuildKit
13
14  Settings
15  - task: DockerInstaller@0
16    displayName: Install Docker
17
18  - script: |
```

Variables

fx

Use variables to store values or encrypted secrets separately from your repository.

New variable

[Learn about variables](#)

Then on New Variable, and add the DOCKERHIB_TOKEN variable, check it as Secret:

← New variable

Name

DOCKERHUB_TOKEN

Value

.....

☒ Keep this value secret

☐ Let users override this value when running this pipeline

To reference a variable in YAML, prefix it with a dollar sign and enclose it in parentheses. For example: `$(DOCKERHUB_TOKEN)`

To use a variable in a script, use environment variable syntax. Replace `.` and space with `_`, capitalize the letters, and then use your platform's syntax for referencing an environment variable. Examples:

Batch script: `%DOCKERHUB_TOKEN%`

PowerShell script: `${env:DOCKERHUB_TOKEN}`

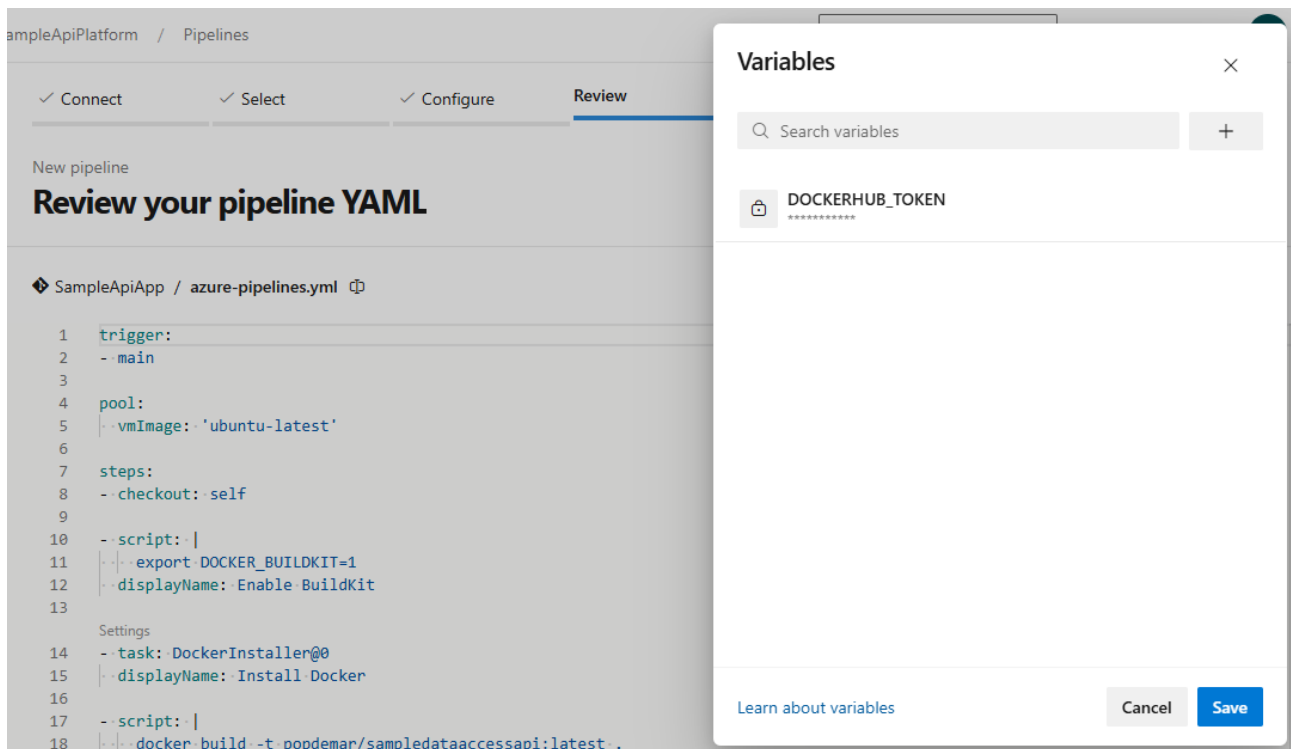
Bash script: `$(DOCKERHUB_TOKEN)`

Click OK
and then
Save

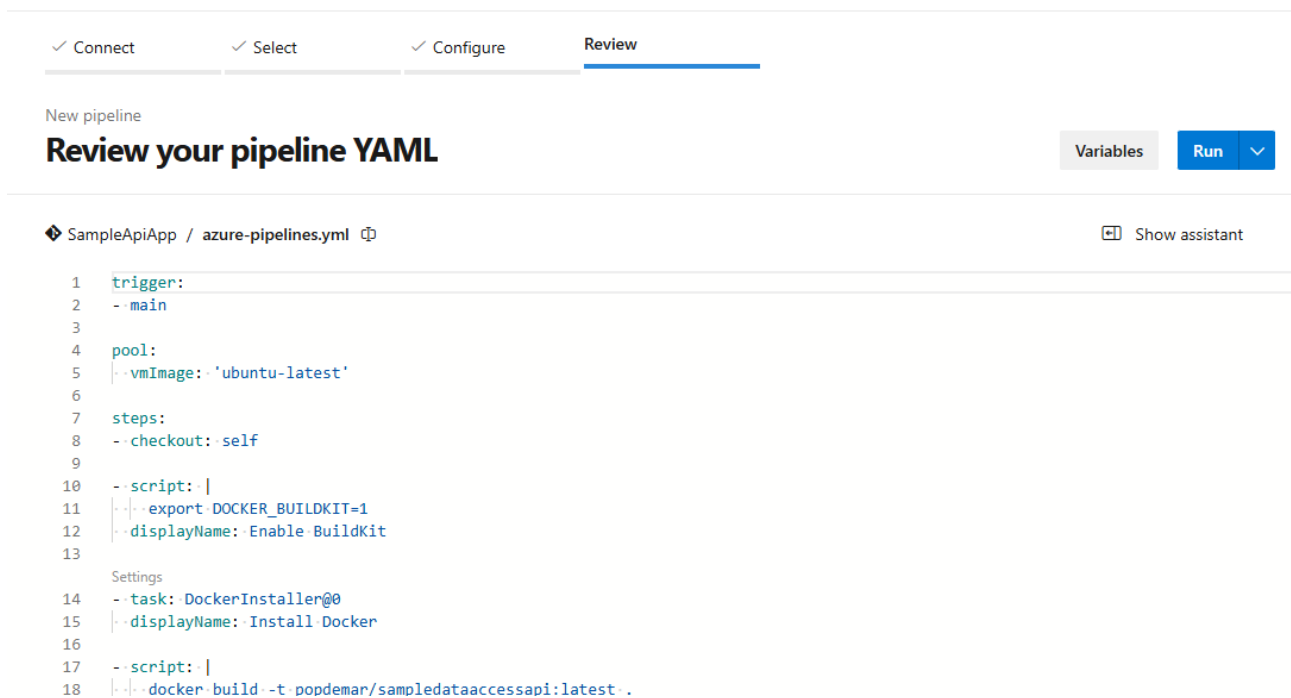
[Learn about variables](#)

Cancel

OK



And now we can run the pipeline: click on the Run button



Once we complete this stage we will have our image, sampleaccessapi, uploaded in the Dockerhub and now we will be able to create a container in Azure.

Repositories

All repositories within the **popdemar** namespace.

Search by repository name

All content

Create repository

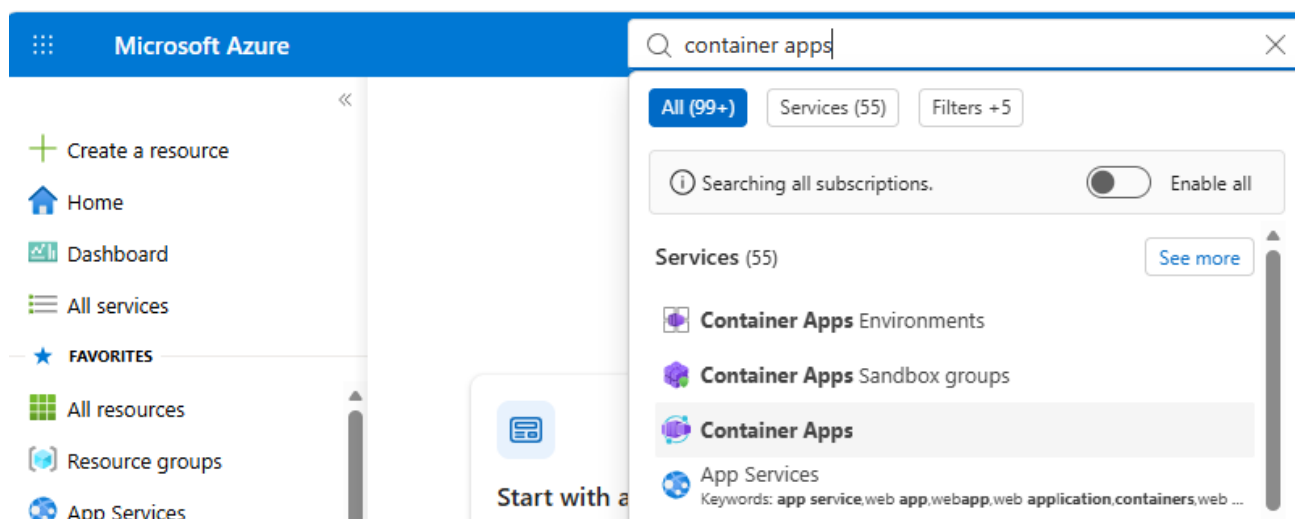
Name	Last Pushed ↑	Contains	Visibility	Scout
popdemar/sampleappapi	less than a minute ago	IMAGE	Public	Inactive
popdemar/sampledataaccessapi	16 minutes ago	IMAGE	Public	Inactive

1-2 of 2<>

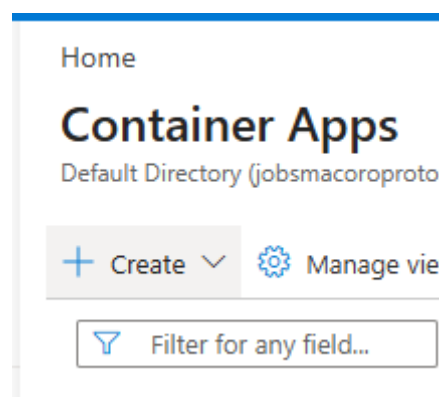
Create Container App in Azure

Before deploying and running our pipeline we need to create in Azure a container in the Container App section as follows:

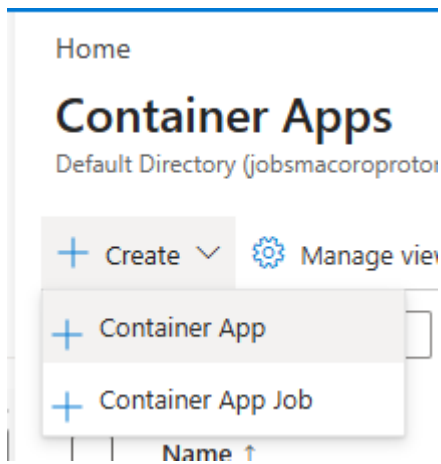
Navigate to Container Apps:



Click on Create:



Select the Container App option:



!!! The resource group should be shared with another container. Look at what we wrote about source groups!!!!

Fill up the form with the following values:

Azure Subscription = your subscription

Resource Group = SampleAppApi_rg (create a new resource group)

Container app name = sampleappapi

Optimize For Azure Functions = False (Unchecked)

Deployment Source = Container Image

Container Apps Environment:

Show Environment In All Regions = False (Unchecked)

Region= France Central (Your Region)

Container Apps Environment= sampleappapi-env (create a new one)

Zone Redundancy = Disabled

Create Container App:

Container Details

Name= sampleappapi

Image Source = Dockerhub or other registries

Image type = public

Registry login server = docker.io

Image and tag = popdemar/sampleappapi:latest

Command override = empty

Arguments override = empty

Development Stack Specific Features:

Development stack = Unspecified

Container Resource Allocation:

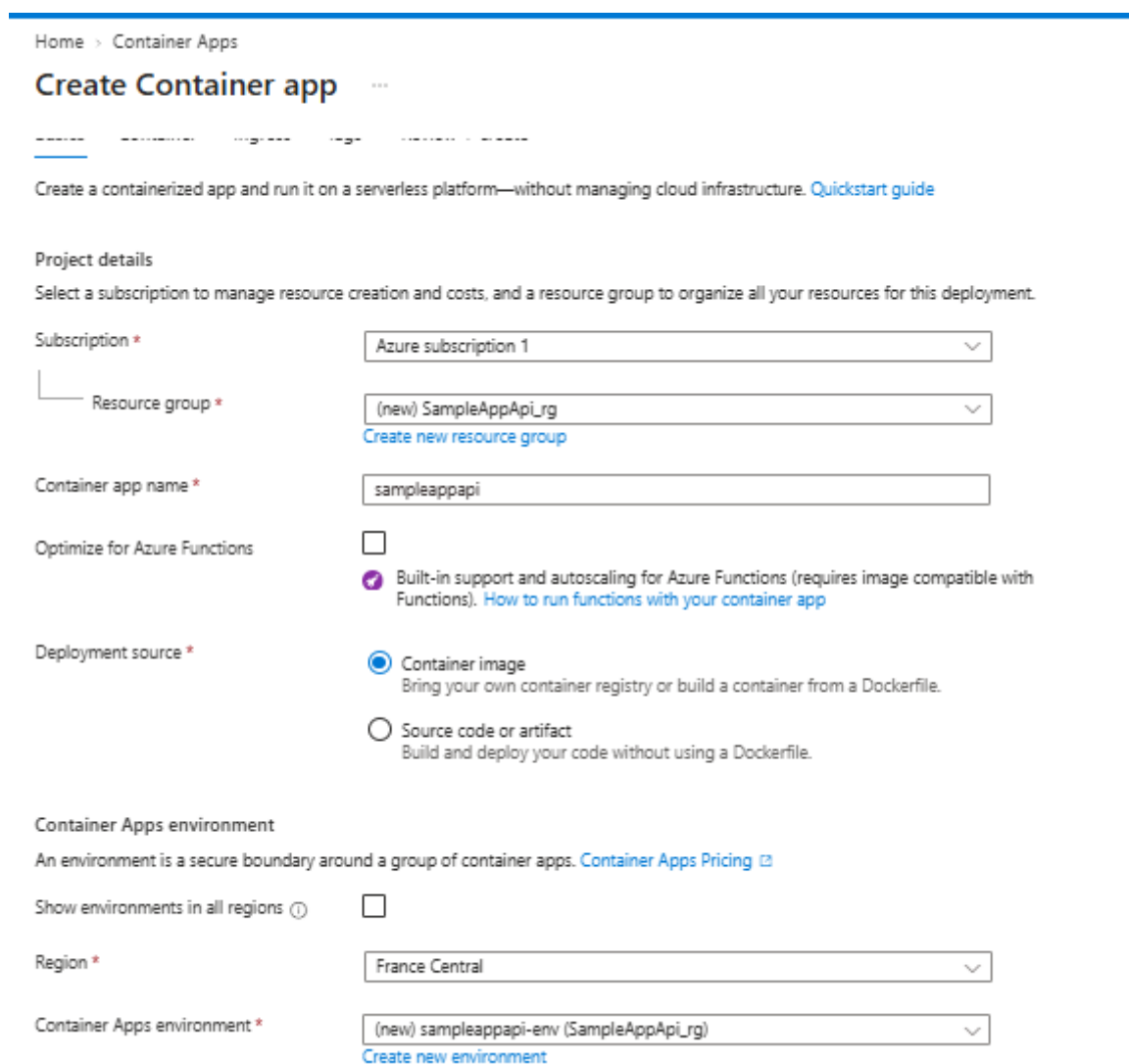
Workload Profile=Consumption-Up to 4 vCPU 8GB memory

GPU = False (Unchecked)

CPU and Memory = 0.5 CPU Cores , 1 Gi memory

Environment Variables:

Keine



The screenshot shows the 'Create Container app' page in the Azure portal. The breadcrumb navigation at the top reads 'Home > Container Apps'. The main heading is 'Create Container app' followed by three dots. Below the heading is a progress bar with five steps: 'Project details', 'Container app', 'Deployment source', 'Container Apps environment', and 'Review + create'. The 'Project details' step is currently active. The instructions state: 'Create a containerized app and run it on a serverless platform—without managing cloud infrastructure. [Quickstart guide](#)'. The 'Project details' section asks to 'Select a subscription to manage resource creation and costs, and a resource group to organize all your resources for this deployment.' The form fields are: 'Subscription *' with a dropdown menu showing 'Azure subscription 1'; 'Resource group *' with a dropdown menu showing '(new) SampleAppApi_rg' and a link 'Create new resource group'; 'Container app name *' with a text input field containing 'sampleappapi'; 'Optimize for Azure Functions' with an unchecked checkbox and a checked radio button for 'Built-in support and autoscaling for Azure Functions (requires image compatible with Functions). [How to run functions with your container app](#)'; 'Deployment source *' with two radio buttons: 'Container image' (selected) with the description 'Bring your own container registry or build a container from a Dockerfile.' and 'Source code or artifact' (unselected) with the description 'Build and deploy your code without using a Dockerfile.'; 'Container Apps environment' section with the text 'An environment is a secure boundary around a group of container apps. [Container Apps Pricing](#) ⓘ'; 'Show environments in all regions ⓘ' with an unchecked checkbox; 'Region *' with a dropdown menu showing 'France Central'; and 'Container Apps environment *' with a dropdown menu showing '(new) sampleappapi-env (SampleAppApi_rg)' and a link 'Create new environment'.

Notice that we have created:

- new resource = SampleAppApi_rg
- new environment = sampleappapi-env

For this purpose fill out the form with the following values:

Environment Name= sampleappapi-env

Zone Redundancy=disabled

Home > Container Apps > Create Container app

Create Container Apps environment ...

Basics Workload profiles Monitoring Networking

i GPU enabled workload profiles are now available. Please note GPU enabled workload profiles can only be added at the time of Azure Container Apps Environment creation.

Environment details

The environment is a secure boundary around one or more container apps that can communicate with each other and share a virtual network, logging, and Dapr configuration. [Learn more](#)

Environment name *

Zone redundancy

A Container App Environment can be deployed as a zone redundant service in the regions that support it. This is a deployment time only decision. You can't make Container App Environment zone redundant after it has been deployed. [Learn more](#)

Zone redundancy *

☒ **Disabled:** Your Container App Environment and the apps in it will not be zone redundant.

☐ **Enabled:** Your Container App Environment and the apps in it will be zone redundant. This requires vNet integration.

Next we have to specify from which image and from which repository this container will be created.

Create Container app ...

① [Create Container Apps with a simplified experience in the new Azure Container Apps portal](#) 

Basics Container Ingress Tags Review + create

Select a quickstart image for your container, or deselect quickstart image to use an existing container.

Use quickstart image ☐

Container details

Name *

Image source
☐ Azure Container Registry
☒ Docker Hub or other registries

Image type
☒ Public
☐ Private

Registry login server * ①

Image and tag *

Command override ①

Arguments override ①

In our case we are storing our image in Dockerhub and our repository is marked in Docker as public.

Development stack-specific features

When you select a specific development stack, you get additional features tailored to that stack—optimizing Container Apps to perform for your unique settings.

Development stack

Container resource allocation

Choose the workload profile for this app. You can adjust the CPU and memory allocation for this app up to the workload profile limit. [Learn more](#)

Workload profile *

GPU ☐

CPU and memory *

Environment variables

Name ▾	Value ▾	Delete
--------	---------	--------

Enter name	Enter value	
---	--	--

Review + create

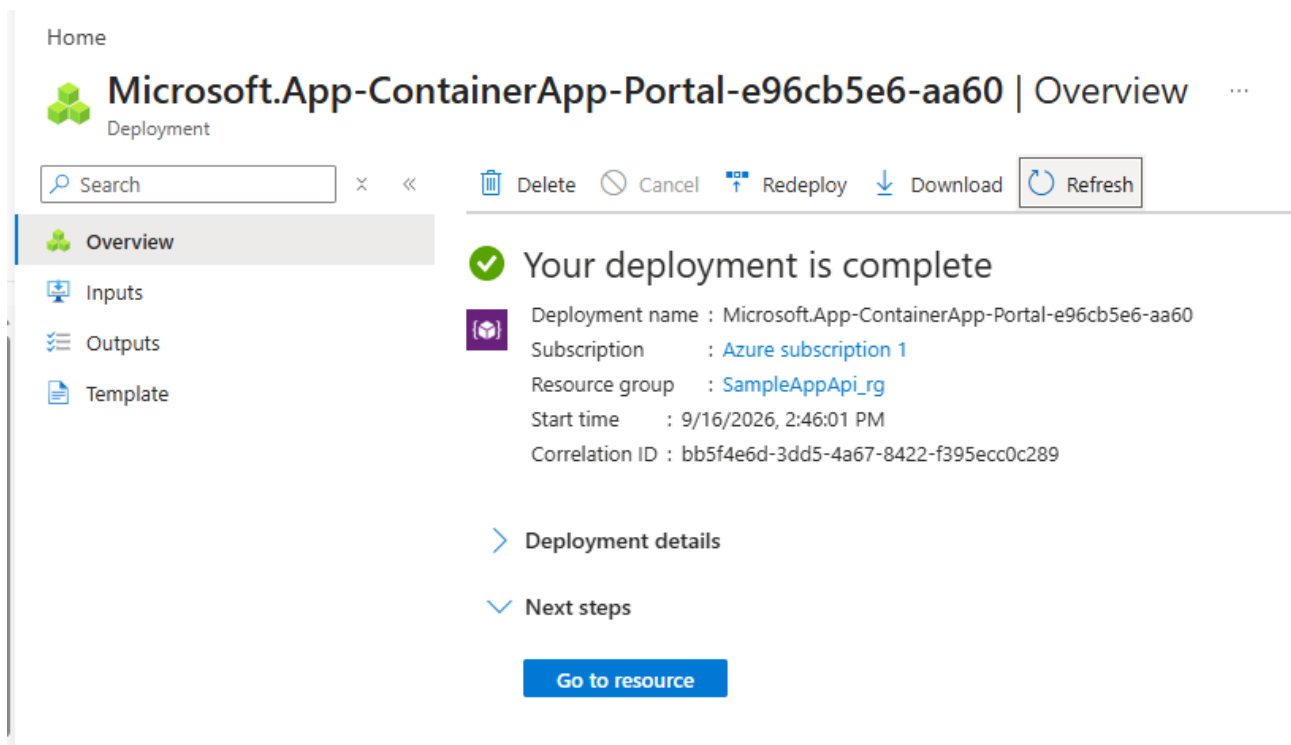
< Previous

Next : Ingress >

The container resource allocation can be left as the default values provided.

In this case we do not need any Environment Variables.

After that click on Review + Create > Create



Once we receive the confirmation that Azure has created our container, we need to check if the container is running, and if there is a revision active, and if the app is reachable etc.

For this purpose in the Azure Portal go to: **Azure Portal** → **Container Apps** → **Deine App** → **Application** → **Revisions**

It should show something like this:

- **Active Revision: Yes**
- **Healthy: Yes**
- **Container: Running**
- **Image: popdemar/<repo>:latest**

When all is green then this means that the deployment was Successful.

Home > Microsoft.App-ContainerApp-Portal-e96cb5e6-aa60 | Overview

sampleappapi ☆ ...
Container App

Search

Stop Refresh Delete Send us your feedback

Overview

- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Resource visualizer
- Application
- Settings
- Networking
- Security
- Monitoring
- Automation
- Help

Running

Location [\(move\)](#)
France Central

Subscription [\(move\)](#)
[Azure subscription 1](#)

Subscription ID
c13b401c-dde4-4298-ad6c-3ef836f847d5

Aspire Dashboard
Not yet active [\(set up\)](#)

Tags [\(edit\)](#)
[Add tags](#)

Container Apps Environment

[sampleappapi-env](#)

Environment type
Workload profiles

Log Analytics
[workspacesampleappapi98](#)

Development stack
Generic [\(manage\)](#)

Revisions with Issues Get started Properties Monitoring

Revisions with Issues ↑	Running status	Running status details	Logs
sampleappapi--q8ssy1o	Failed	1/1 Container crashing: sampleappapi	Show logs

Add or remove favorites by pressing Ctrl+Shift+F

However in our case it was not green , but red. When trying to get the logs did not give us too much information.

az containerapp logs show \

--name sampleappapi \

--resource-group SampleAppApi_rg \

--follow

We can also get the system logs:

az containerapp logs show \

--name sampleappapi \

--resource-group SampleAppApi_rg \

--type system

After removing the all the code in Program.cs related to the JWT (the key was missing) and redeploying the image with the pipeline and updating the image from the command line, we managed to create the container.

az containerapp update \

--name sampleappapi \

--resource-group SampleAppApi_rg \

--image popdemar/sampleappapi:latest

And after the problem has been solved and the container is up and running:

Home > sampleappapi

sampleappapi | Revisions and replicas

Container App

Search

+ Create new revision Save Refresh Deployment mode Send us your feedback

Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Resource visualizer

Application

- Revisions and replicas
- Containers
- Scale
- Volumes

Now you can manage this application in the [Azure Container Apps portal](#).

Traffic routing and labels are only supported when Ingress is enabled. To enable Ingress for your app, go to the [Ingress page](#).

Each revision is an immutable snapshot of your container app, and can have different setups for traffic allocation, container images, autoscaling, or Dapr. Make updates to your app by creating a new revision. [Learn more](#)

Search

Active revisions Inactive revisions Replicas

Name ↑ ↓	Date created ↓	Running status ↓	View Logs ↓	Label ↓	Traf
sampleappapi--0000001	18.09.2026, 03:...	Running (at max) ✓	Show Logs		0
sampleappapi--j34e2t5	17.09.2026, 17:...	Failed ✗	View details Show Logs		0

Finally now we will run the normal pipeline that deploys the code. Before we could not do it because the azure devops was complaining that the container did not exist:

Pipeline

Now we can deploy the normal pipeline that will deploy the code and update the image. Before we could not do it because the container did not exist yet in Azure, and then Azure DevOps cannot complete the job because it cannot find the container:

trigger:

- master

pool:

vmImage: ubuntu-latest

variables:

- name: solution

value: '**/*.*.sln'

- name: buildPlatform

value: 'Any CPU'

- name: buildConfiguration

value: 'Release'

#this variable is necessary to allow the agent to give access to the agent to the NuGet packages.

- name: NUGET_PACKAGES

value: \$(Agent.TempDirectory)/.nuget/packages

steps:

Restore

- task: DotNetCoreCLI@2

```
displayName: Restore packages
inputs:
  command: restore
  projects: '$(solution)'

# -----
# Build
# -----
- task: DotNetCoreCLI@2
  displayName: Build solution
  inputs:
    command: build
    projects: '$(solution)'
    arguments: '--configuration $(buildConfiguration)'

# -----
# Test + Coverage
# -----
- script: |
  dotnet test SampleAppApi.sln --configuration Release --collect:"XPlat Code Coverage"
  displayName: Run tests with coverage

# -----
# Publish coverage
# -----
- task: PublishCodeCoverageResults@1
  displayName: Publish coverage
  inputs:
    codeCoverageTool: Cobertura
    summaryFileLocation: '$(Agent.TempDirectory)**/coverage.cobertura.xml'
    reportDirectory: '$(Agent.TempDirectory)'

# -----
# Build Docker image
# -----
- task: Docker@2
  displayName: Build Docker image
  inputs:
    command: build
    Dockerfile: Dockerfile
    repository: '$(DOCKERHUB_USERNAME)/sampleappapi'
    tags: |
      latest

- script: |
  echo "$(DOCKERHUB_TOKEN)" | docker login -u "$(DOCKERHUB_USERNAME)" --
  password-stdin
  displayName: Docker Hub Login

# -----
# Push Docker image
# -----
```

```

- script: |
  docker push $(DOCKERHUB_USERNAME)/sampleappapi:latest
  displayName: Push Docker image

# -----
# Update ACA image
# -----
- task: AzureCLI@2
  displayName: Update ACA image
  inputs:
    #we reuse the same service connection as in SampleDataAccessApi because it is only to login
    into Azure
    azureSubscription: 'SampleAppApi_ServiceConnection'
    scriptType: bash
    scriptLocation: inlineScript
    inlineScript: |
      az containerapp update \
        --name sampleappapi \
        --resource-group SampleAppApi_rg \
        --image docker.io/$(DOCKERHUB_USERNAME)/sampleappapi:latest

```

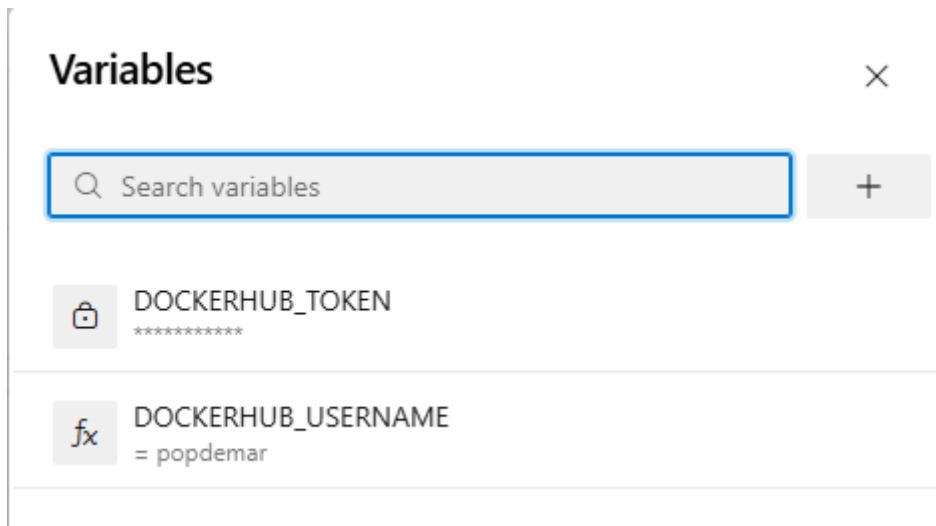
We can reuse the service connection for Docker that we have previously created for the SampleDataAccessApi: DockerHub_ServiceConnection. However, we will need to create an new ARM Service Connection (SampleAppApi_ServiceConnection) as we did for the previous container app because the usage of that service connection is limited to the resource group that is not the same as the one that we are currently using: as well as the SampleDataAccessApi_ServiceConnection that we had used in the previous pipeline to allow Azure Devops to connect to Azure and execute the command to update the image. This reuse is valid, however the name should be more appropriate:

```

azureSubscription: 'SampleAppApi_ServiceConnection'
scriptType: bash
scriptLocation: inlineScript
inlineScript: |
  az containerapp update \
    --name sampleappapi \
    --resource-group SampleAppApi_rg \
    --image docker.io/$(DOCKERHUB_USERNAME)/sampleappapi:latest

```

Do not forget to check that the following pipeline variables are still there: DOCKERHUB_TOKEN, DOCKERHUB_USERNAME.



Managed Identity

Now we go to the Container App and we will see this User Defined Identity in the list of available identities that we can use:

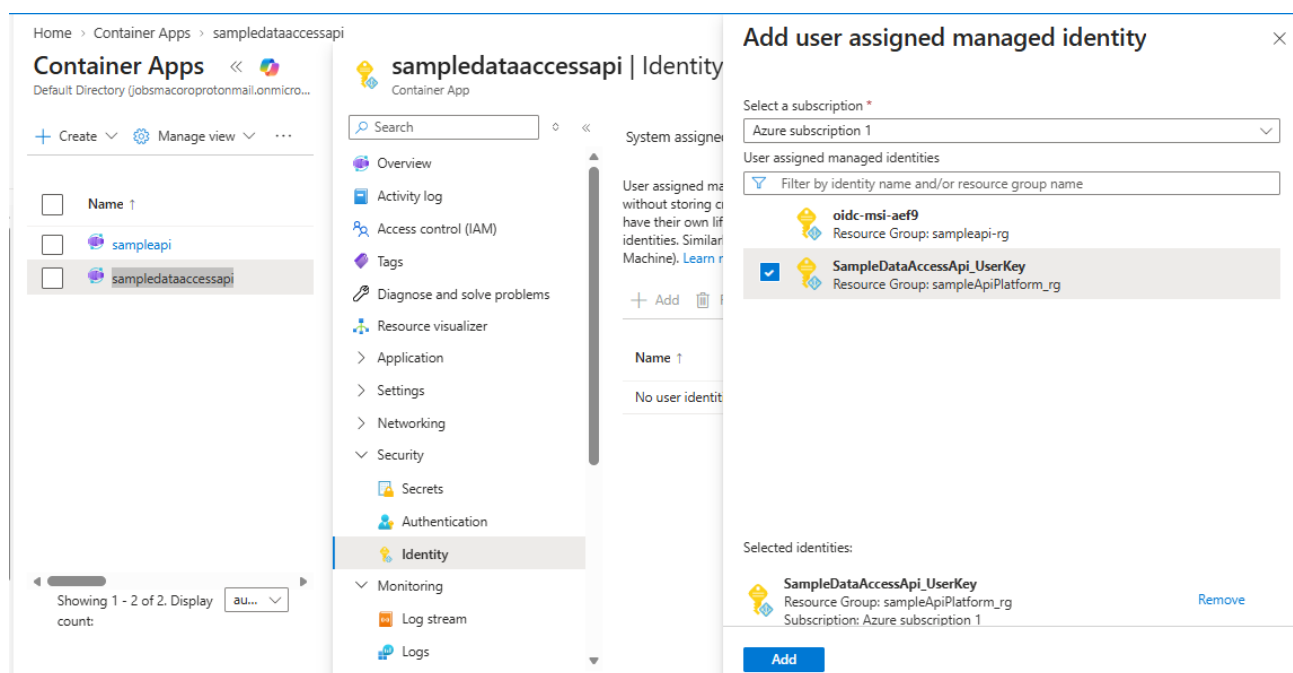


Figure 25: Adding The User Defined Identity To The Container App

After clicking on the Add button:

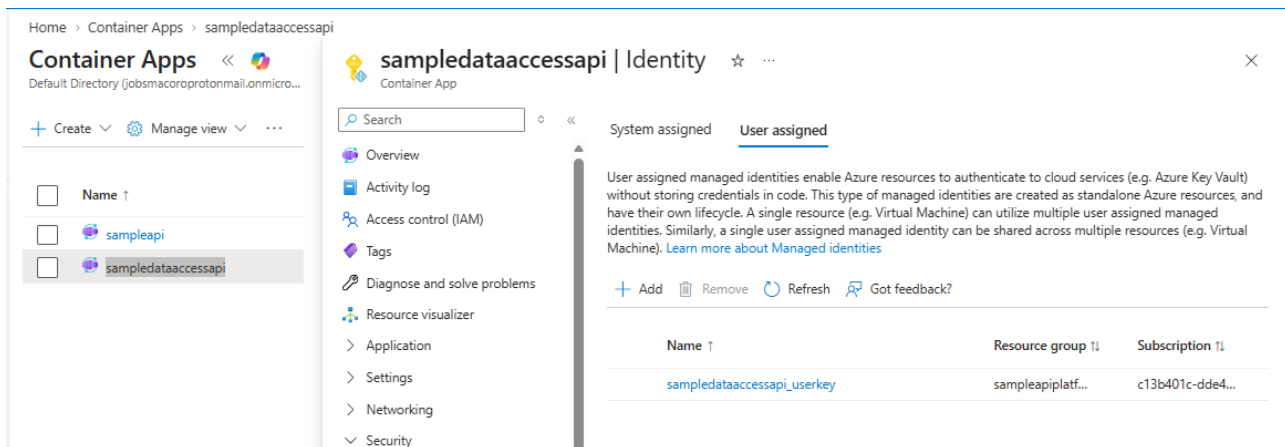


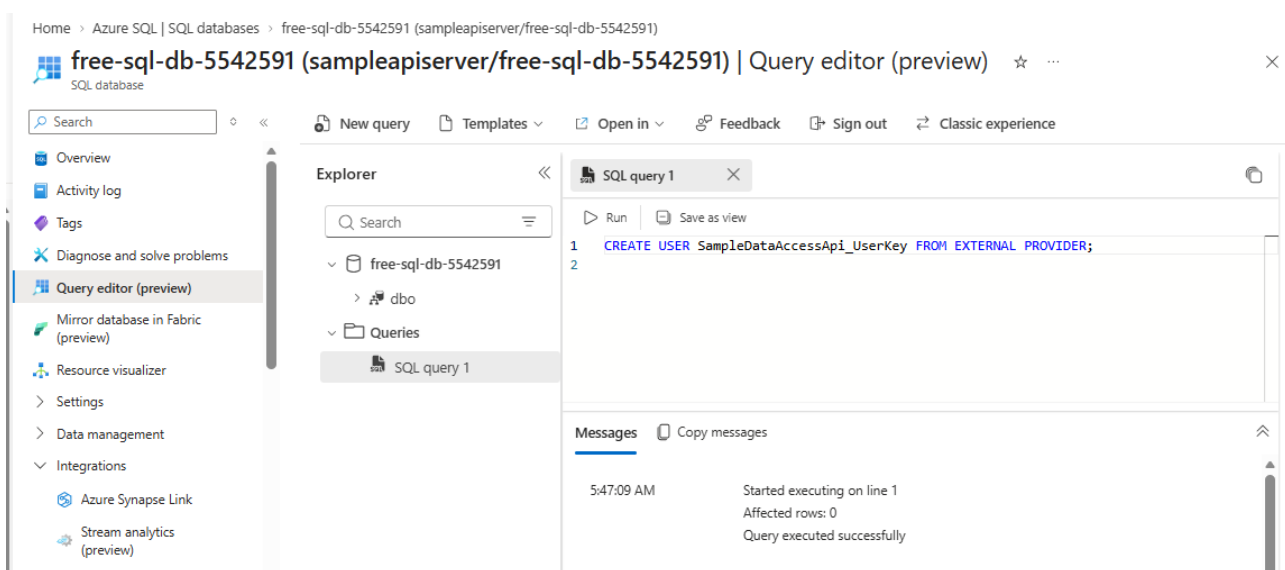
Figure 26: User Defined Identity Added To The Container App

Identity Roles

Now we need to add permissions to this User Identity: for instance to give access to our Azure SQL Database.

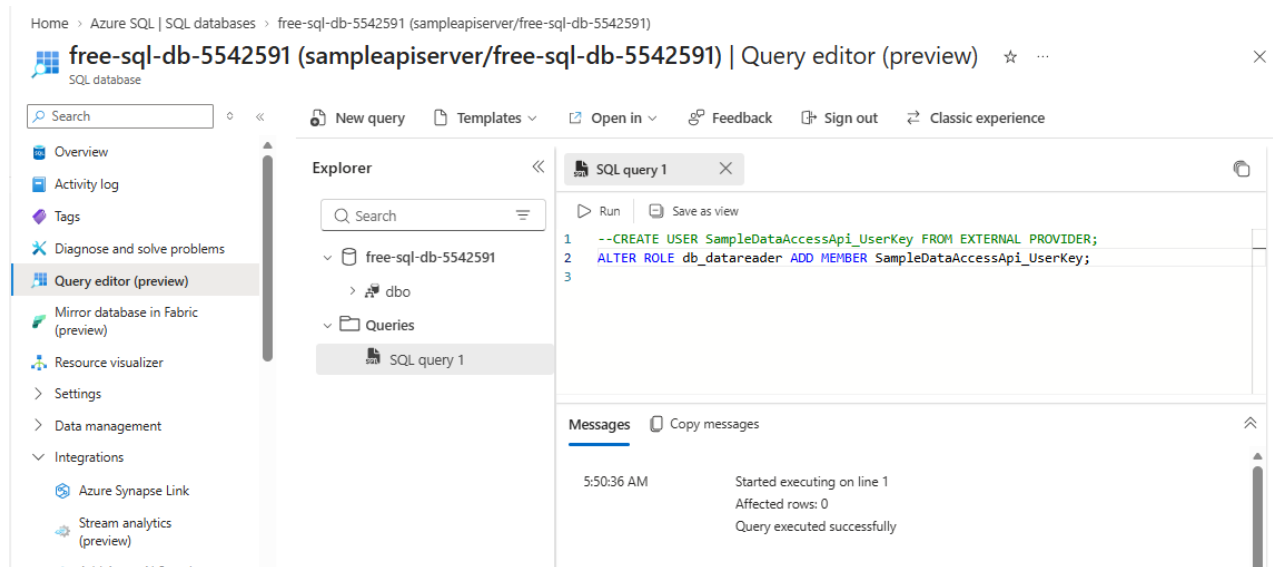
We need to login into our Azure Database with an Entra ID admin credential (in our case as jobs.macoro), not as a database admin (sampleapiserveradmin), and add our User Defined Identity , SampleDataAccessApi_UserKey, so that the database recognizes as a user and can login without a password (behind the scenes Azure will exchange a token between the database and the container).

CREATE USER SampleDataAccessApi_UserKey FROM EXTERNAL PROVIDER;

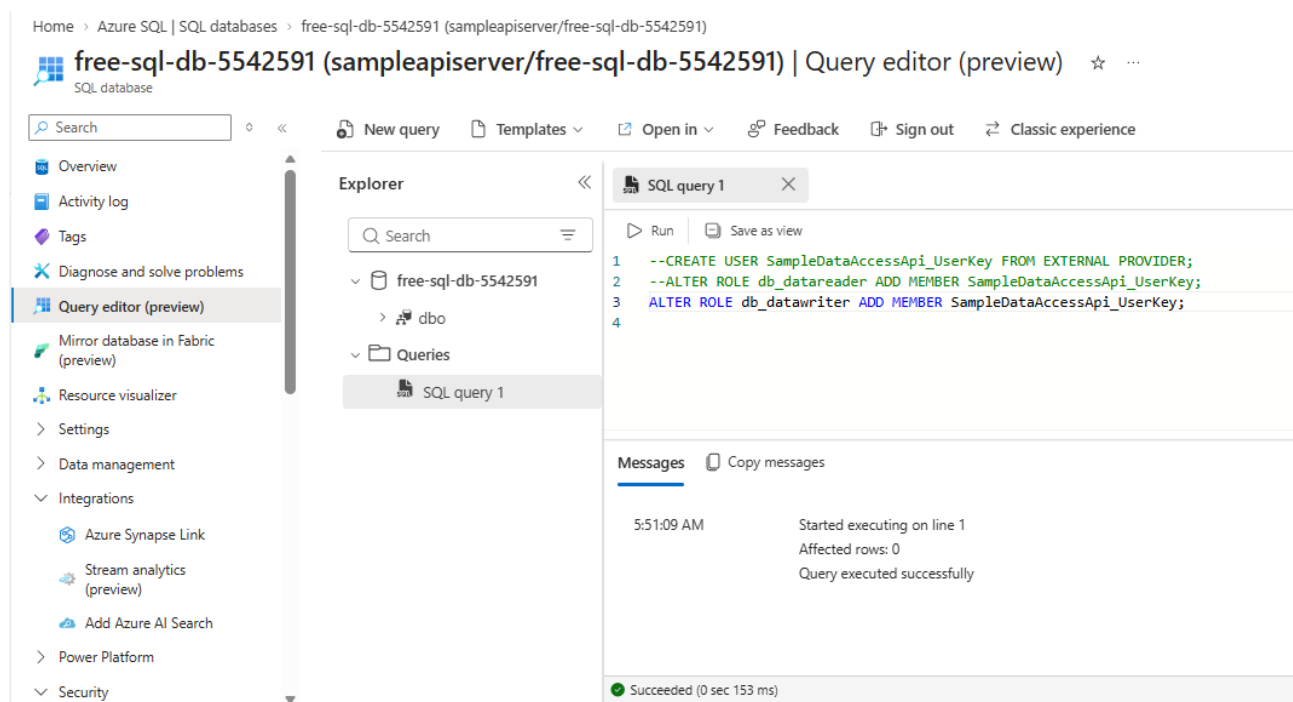


and that allows to login without password and to performed the operations.

ALTER ROLE db_datareader ADD MEMBER SampleDataAccessApi_UserKey;

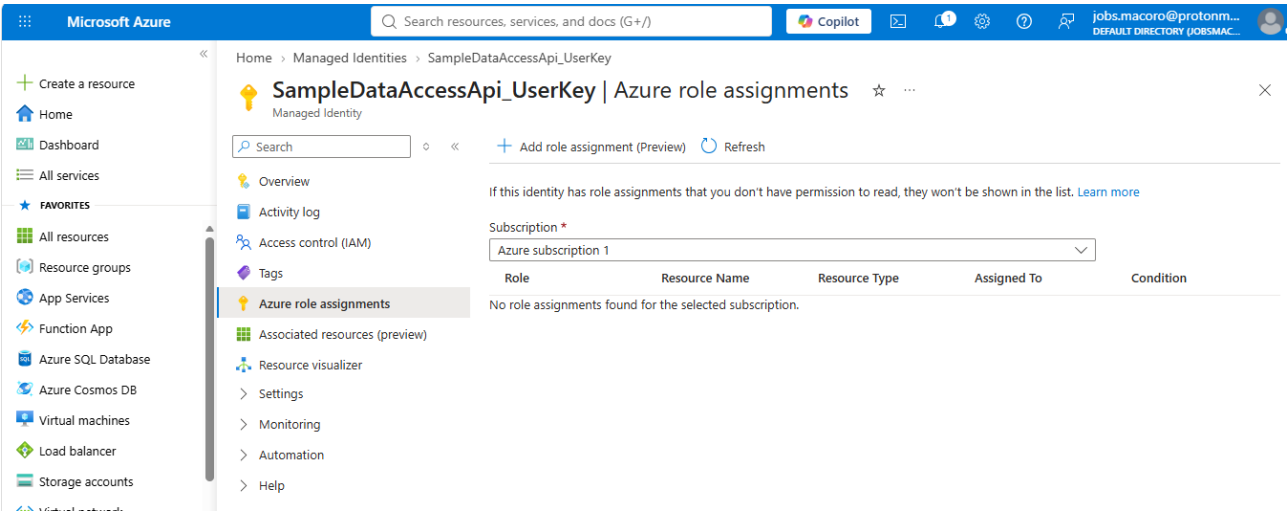


ALTER ROLE db_datawriter ADD MEMBER SampleDataAccessApi_UserKey;



Here are the tasks that we need to perform are similar to the ones that we did for the SampleDataAccessApi.

In order to define roles for our identity in Azure we can do it here: Azure Portal > Managed Identities > *SampleDataAccessApi_UserKey* > Azure role assignments



Then we click on the + Add Role Assignment button

A role can be defined at four different scopes: Subscription, Resource Group, Key Vault, Storage, SQL.

In our application currently we have the following defined.

Object	Object Type	Resource Group	Identity Roles
Subscription	Subscription		Contributor
sampleappapi	Container app	SampleAppApi_rg	SampleAppApi_rReader.
sampledataaccessapi	Container app	SampleDataAccessApi_rg	SampleDataAccessApi_rReader
sampledataaccessapi-env2	Resource Container app Enviroment for sampledataaccessapi	sampleDataAccessApi_rg	Inherits from the resource group or subscription where the Container app has been defined. In this case will be Contributor or Reader.
sampleappapi-env5	Resource Container app Enviroment	SampleAppApi_rg	Inherits from the resource group or subscription where the

	for sampledataa ccessapi	Container app has been defined. In this case will be Contributor or Reader.
free-sql-db-5542591 (sampleapiserver/free-sql-db-5542591)	SQL Database	DefaultResourceGroup-CCAN db_datareader db_datawriter
Key Vault(in case we save secrets)		Key Vault Secrets User
Log Analytics Workspace		Reader
Storage		Storage Blob Data Reader

We will define the Identity Roles defined in the correspondent column for the resources / objects that are in bold.

For the ACA environments: sampledataaccessapi-env2, sampleappapi-env5, they will inherit the role from the Subscription or from the resource group of the container. As we have defined roles for both, the subscription and the resource groups of each container app, in this case they will take the role from the container app resource group as it overwrites the roles defined in at Subscription level.

We have defined this user key in the following resource group: sampleApiPlatform_rg

For the Log Analytics Workspace, Storage and Key Vault we will only be able to define a role once we have one.

At the end of this process we will get all the roles defined:

Home > Managed Identities > SampleDataAccessApi_UserKey

SampleDataAccessApi_UserKey | Azure role assignments

Search Add role assignment (Preview) Refresh

If this identity has role assignments that you don't have permission to read, they won't be shown in the list. [Learn more](#)

Subscription *
Azure subscription 1

Role	Resource Name	Resource Type	Assigned To	Condition
Reader	SampleAppApi_rg	Resource Group	SampleDataAccessApi_Userkey	None
Reader	sampleDataAccessApi_rg	Resource Group	SampleDataAccessApi_Userkey	None
Contributor	Azure subscription 1	Subscription	SampleDataAccessApi_Userkey	None

Communication between Containers

We can use : Inress/HTTPS or Dapr.

Dapr requires that all of the containers use the same ACA environment which is not the case. Now we have:

Container App	Environment
sampledataaccessapi	Sampledataaccessapi-env2
sampleappapi	sampleappapi-env5

For this reason we will need to recreate all of the container apps again and both having the same Environment. I have deleted the previous containers and created new ones

Container App	Environment
sampledataaccessapi	sampleapi-env
sampleappapi	sampleapi-env

We will follow the same steps as described in previous sections and expecting not to encounter the same issues as now we know that the images are have been into ACA successfully deployed and once created we will assign them the sampleDataAccessApi_UserKey User assigned role to the both of them.

We don not need to change the roles as the reource groups where these new containers are located is the same as the previous ones, and the ACA environments will inherit the roles from these resource groups as we have mentioned before.

Dapr Architecture

Dapr consists of two parts:

Your Container (App)

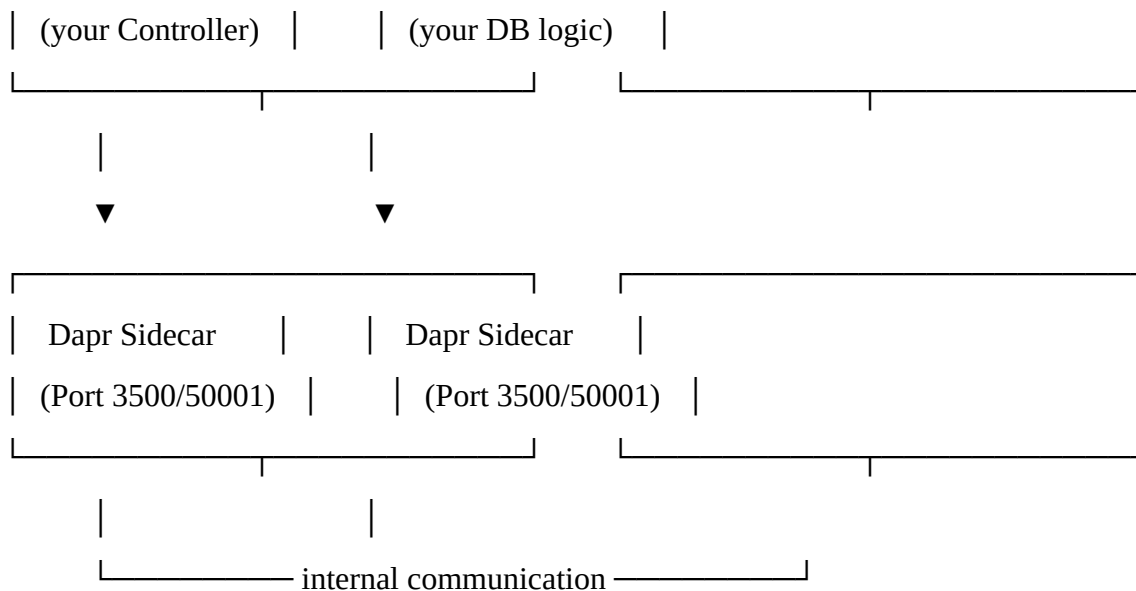
The Dapr Sidecar (always runs alongside it)

Together, both form a single unit.

1.Architecture: SampleAppApi → Dapr → SampleDataAccessApi This is how the communication looks:

Code





No URLs

No Ports

No Ingress

Only internal sidecar communication

2.Architecture: SampleDataAccessApi → Database Code SampleDataAccessApi → EF Core
→ SQL Server / Azure SQL

Dapr is NOT involved here.

Dapr does NOT replace EF Core.

Dapr is NOT a database proxy.

3.Why only SampleAppApi needs the Dapr Client **SampleAppApi** calls other container apps

needs service discovery

needs sidecar invocation

needs the Dapr client or sidecar HTTP

SampleDataAccessApi only receives requests

does NOT need to call other services

does NOT need to use the Dapr client

does NOT need to use Dapr components

does NOT need to use Dapr state store

It only needs the sidecar so that it is reachable.

4.Dapr Service Invocation Flow This is how a request works: SampleAppApi calls:
`http://localhost:3500/v1.0/invoke/dataaccess/method/movies`

Dapr sidecar of SampleAppApi:

automatically finds the correct revision

automatically finds the correct container app

handles automatic retries

handles automatic circuit breakers

handles automatic telemetry

Dapr sidecar of SampleDataAccessApi:

receives the request

forwards it to your API

your API executes EF Core

the response goes back via the sidecar

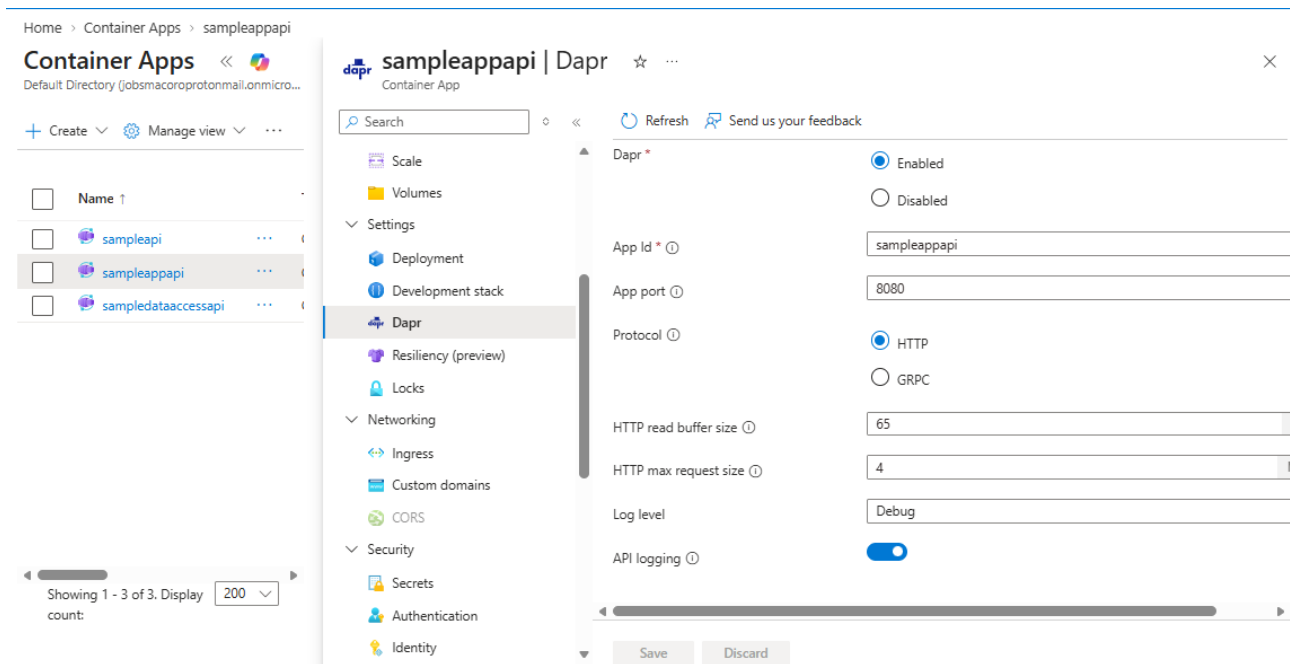
You don't need to know any URLs.

You don't need to know any ports.

You don't need to know any ingress endpoints.

5.Why Dapr is perfect for your architecture Microservices without ingress Zero-trust communication No ports, no DNS issues Automatic service discovery Automatic retries Automatic circuit breakers Automatic observability Perfect for Azure Container Apps

Setting Dapr for SampleAppApi



Dapr Settings = Enabled

App Id = sampleappapi

App Port = 8080

Protocol = HTTP (because our APIs are HTTP Rest)

HTTP read buffer size = 65 KB (this is the maximum value value.)

HTTP max request size = 4 MB (This is the standard value. And it is the max size HTTP request that the sidecar accept)

Log Level = Debug (for test) / Info (for production)

API Logging = On for testing / Off for production.. It will log all Dapr operations. For production set it to off, to avoid to generate too many logs.

Setting Dapr for SampleDataAccessApi

Dapr Settings = Enabled

App Id = sampledataaccessapi

App Port = 8080

Protocol = HTTP (because our APIs are HTTP Rest)

HTTP read buffer size = 65 KB (this is the maximum value value.)

HTTP max request size = 4 MB (This is the standard value. And it is the max size HTTP request that the sidecar accept)

Log Level = Debug (for test) / Info (for production)

API Logging = On for testing / Off for production.. It will log all Dapr operations. For production set it to off, to avoid to generate too many logs.

Install the NuGet package for Dapr: `dotnet add package Dapr.AspNetCore`

Activate the dapr client in the SampleAppApi in Program.cs: `builder.Services.AddDaprClient();`

In Program.cs add the following code. The header gives the instruction to the sidecar to provide an answer in less than 30 seconds.

```
builder.Services.AddHttpClient<IDataAccessClient, DataAccessClientDapr>(client =>
{
    client.BaseAddress = new Uri("http://localhost:3500");
    //instruct the dapr sidecar to return an response in less than 504. Otherwise timeout.
    client.DefaultRequestHeaders.Add("Dapr-TimeoutInSeconds", "30");
});
```

Make sure that:

- to remove [Authorize] from sampledataaccessapi movies controller as it is no needed anymore in the context of dapr are the connections between containers are managed by azure itself (trusted)

- make sure the url to connect to the sampledataaccessapi container in dapr is correct, according to the path defined in the controller:

```
[Route("api/movies")]
```

```
v1.0/invoke/sampledataaccessapi/method/api/movies
```

- Make sure that all interfaces and the classes that implement that interface are declared in the Program.cs:

```
builder.Services.AddScoped<IMovieRepository, MovieRepository>();
builder.Services.AddScoped<IMovieService, MovieService>();
```

-

And then replace the class that queries that sampledataaccess container by one that is dapr compliant:

```
using NuGet.SampleSharedModels.DTO;
using NuGet.SampleSharedModels.Results;
using SampleAppApi.Interfaces.ExternalServices;
using Microsoft.Extensions.Logging;
using System.Net.Http.Json;
using NuGet.SampleSharedModels.Interfaces;
```

```
public class DataAccessClientDapr : IDataAccessClient
{
    private readonly HttpClient _httpClient;
    //private readonly ISharedServicesClient _sharedServicesClient;
    private readonly ILogger<DataAccessClientDapr> _logger;
    private readonly string _category = string.Empty;
```

```

public DataAccessClientDapr(IHttpClientFactory httpClientFactory,
                           ILogger<DataAccessClientDapr> logger
                           //ISharedServicesClient sharedServicesClient
                           )
{
    // Dapr Sidecar HTTP Port ist IMMER 3500 in Azure Container Apps
    _httpClient = httpClientFactory.CreateClient("dapr");
    //_sharedServicesClient = sharedServicesClient;
    _logger = logger;
    _category = this.GetType().Name;
}

public async Task<ServiceResult<IEnumerable<MovieDTORead>>> GetAll()
{
    try
    {
        var url = "v1.0/invoke/dataaccess/method/movies";

        var result = await
        _httpClient.GetFromJsonAsync<ServiceResult<IEnumerable<MovieDTORead>>>(url);

        return result!;
    }
    catch (Exception ex)
    {
        var message = $"DataAccessApi unreachable: {ex.Message}";
        _logger.LogError("GetAll Exception: {message}", message);
        //await _sharedServicesClient.LogAsync(_category, $"GetAll Exception: {message}",
LogLevel.Error);
        return ServiceResult<IEnumerable<MovieDTORead>>.Fail(new List<string> { message });
    }
}

public async Task<ServiceResult<MovieDTORead>> GetById(string id)
{
    try
    {
        var url = $"v1.0/invoke/dataaccess/method/movies/{id}";

        var result = await _httpClient.GetFromJsonAsync<ServiceResult<MovieDTORead>>(url);

        return result!;
    }
    catch (Exception ex)
    {
        var message = $"DataAccessApi unreachable: {ex.Message}";
        _logger.LogError("GetById Exception: {message}", message);
        //await _sharedServicesClient.LogAsync(_category, $"GetById Exception: {message}",
LogLevel.Error);
        return ServiceResult<MovieDTORead>.Fail(new List<string> { message });
    }
}

```

```

}

public async Task<ServiceResult<MovieDTORead>> Add(MovieDTOAdd movieDTOAdd)
{
    try
    {
        var url = "v1.0/invoke/dataaccess/method/movies";

        var response = await _httpClient.PostAsJsonAsync(url, movieDTOAdd);

        var result = await
response.Content.ReadFromJsonAsync<ServiceResult<MovieDTORead>>();

        return result!;
    }
    catch (Exception ex)
    {
        var message = $"DataAccessApi unreachable: {ex.Message}";
        _logger.LogError("Add Exception: {message}", message);
        //await _sharedServicesClient.LogAsync(_category, $"Add Exception: {message}",
LogLevel.Error);
        return ServiceResult<MovieDTORead>.Fail(new List<string> { message });
    }
}

public async Task<ServiceResult<MovieDTORead>> Update(string id, MovieDTOUpdate
movieDTOUpdate)
{
    try
    {
        var url = $"v1.0/invoke/dataaccess/method/movies/{id}";

        var response = await _httpClient.PutAsJsonAsync(url, movieDTOUpdate);

        var result = await
response.Content.ReadFromJsonAsync<ServiceResult<MovieDTORead>>();

        return result!;
    }
    catch (Exception ex)
    {
        var message = $"DataAccessApi unreachable: {ex.Message}";
        _logger.LogError("Update Exception: {message}", message);
        //await _sharedServicesClient.LogAsync(_category, $"Update Exception: {message}",
LogLevel.Error);
        return ServiceResult<MovieDTORead>.Fail(new List<string> { message });
    }
}

public async Task<ServiceResult<bool>> Delete(string id)
{
    try

```

```

{
    var url = $"v1.0/invoke/dataaccess/method/movies/{id}";

    var response = await _httpClient.DeleteAsync(url);

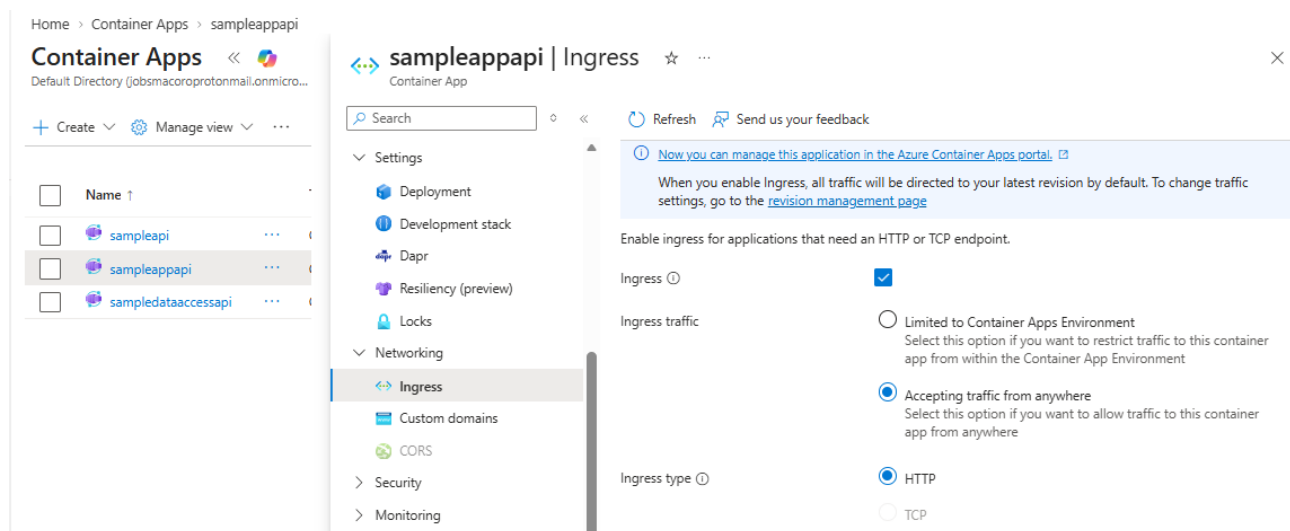
    var result = await response.Content.ReadFromJsonAsync<ServiceResult<bool>>();

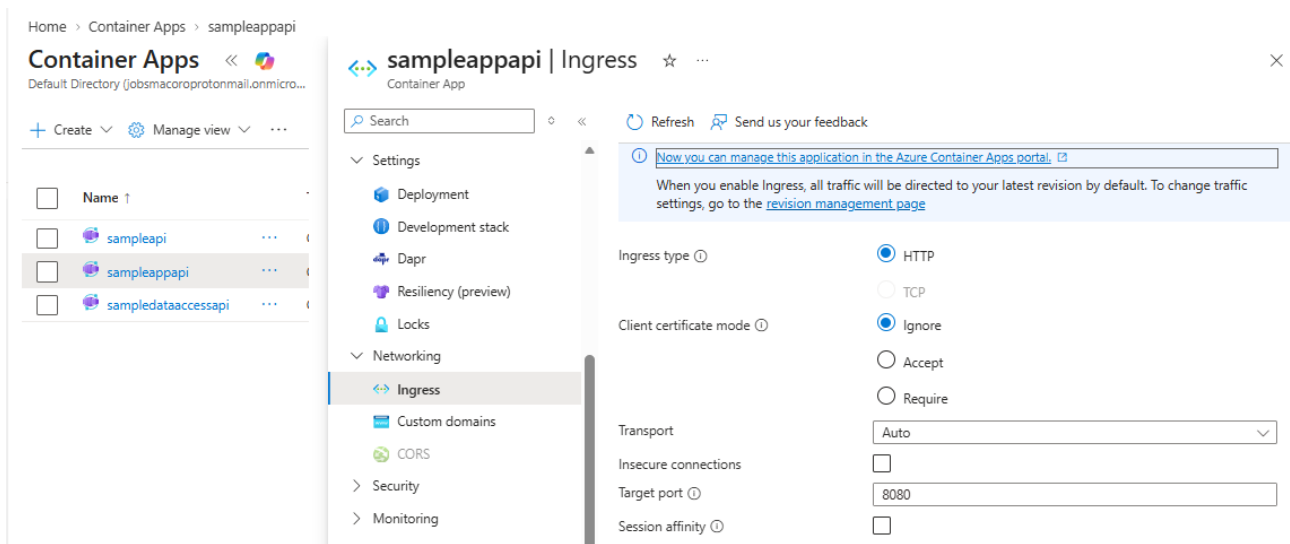
    return result!;
}
catch (Exception ex)
{
    var message = $"DataAccessApi unreachable: {ex.Message}";
    _logger.LogError("Delete Exception: {message}", message);
    //await _sharedServicesClient.LogAsync(_category, $"Delete Exception: {message}",
LogLevel.Error);
    return ServiceResult<bool>.Fail(new List<string> { message });
}
}
}

```

Ingress activation for the SampleAppApi

This is necessary when our application needs to be called by client applications: Angular UI, Postman, Browser, Mobile App, Swagger, Azure Devops Health Checks





Additional TCP ports

IP Restrictions

Access restrictions allow you to define lists of allow/deny rules to control traffic to your app. If there are no rules defined then your app will accept traffic from any address.

IP Security Restrictions Mode

- ☒ Allow all traffic (default)
- ☐ Allow traffic from IPs configured below, deny all other traffic
- ☐ Deny traffic from IPs configured below, allow all other traffic

Ingress traffic= Accepting Traffic from Anywhere

Ingress Type = HTTP (we are exposing a REST interface)

Client Certificate Mode: Ignore (we do not need any mTLS for an open API)

Transport: Auto (azure will decide between HTTP/1 or HTTP/2)

Insecure connections=False (You want to enforce HTTPS)

Target port=8080

Session affinity= False (We do not have any Session based workloads)

IP Restrictions > IP Security Restrictions Mode = Allow all traffic (Default) (To make the API reachable from anywhere).

Machine-to-Machine JWT

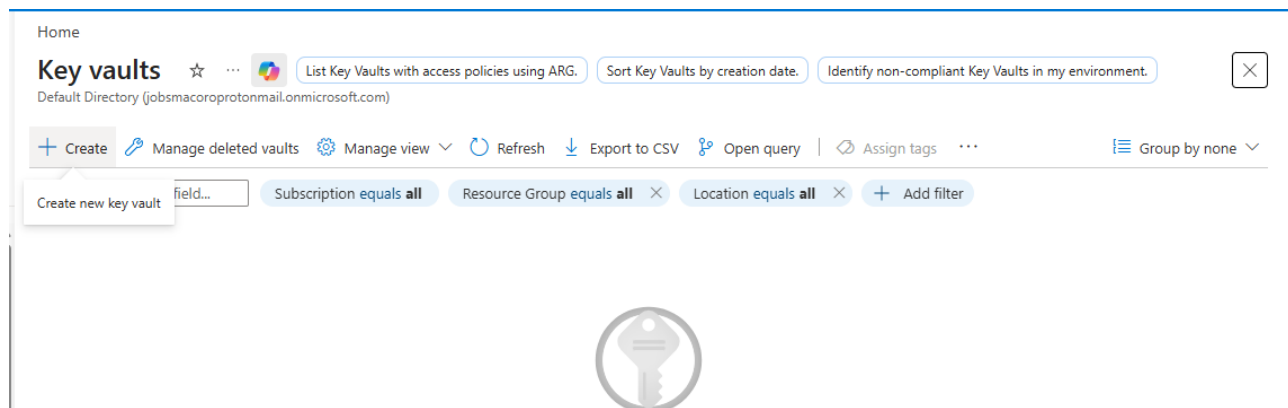
We will require all the applications that want to access the SampleAppApi to provide a Service Token.

This tokens will be stored in the Key Vault.

This requires the following steps

Create A Key Vault

Portal > Search: Key Vaults > Click on on the “+ Create” button



Resource Group = sampleApiPlatform_rg (because the Key Vault belongs logically to the whole platform)

Instance Details:

Key vault name = sampleapiplatform-kv

Region = France Central (in the same region where we have our containers for performance and compliance).

Pricing Tier = Standard (Premium is basically for HSM = Hardware Security Module which we do not need)

Recovery Option

Soft delete = enable (in case that we delete a secret that we should not have deleted)

Days to retain deleted vaults = 90 days

Purge Protection = Disable purge protection (allow key vault and objects to be purged during retention period) . For demos this is the right option. If enabled the vault will be not possible to delete for 90 days (days to retain deleted vaults).

Create a key vault ...

Basics Access configuration Networking Tags Review + create

Azure Key Vault is a cloud service used to manage keys, secrets, and certificates. Key Vault eliminates the need for developers to store security information in their code. It allows you to centralize the storage of your application secrets which greatly reduces the chances that secrets may be leaked. Key Vault also allows you to securely store secrets and keys backed by Hardware Security Modules or HSMs. The HSMs used are Federal Information Processing Standards (FIPS) 140-3 Level 3 validated. In addition, key vault provides logs of all access and usage attempts of your secrets so you have a complete audit trail for compliance.

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Azure subscription 1

Resource group *

sampleApiPlatform_rg

[Create new](#)

Create a key vault ...

Instance details

Key vault name * ⓘ

sampleapiplatform-kv ✓

Region *

France Central

Pricing tier * ⓘ

Standard

Create a key vault ...

Recovery options

Soft delete protection will automatically be enabled on this key vault. This feature allows you to recover or permanently delete a key vault and secrets for the duration of the retention period. This protection applies to the key vault and the secrets stored within the key vault.

To enforce a mandatory retention period and prevent the permanent deletion of key vaults or secrets prior to the retention period elapsing, you can turn on purge protection. When purge protection is enabled, secrets cannot be purged by users or by Microsoft.

Soft-delete ⓘ

Enabled

Days to retain deleted vaults * ⓘ

90

Purge protection ⓘ

- ☒ Disable purge protection (allow key vault and objects to be purged during retention period)
- ☐ Enable purge protection (enforce a mandatory retention period for deleted vaults and vault objects)

Previous

Next

Review + create

Click on Review and create and the Key Vault will be created:

Home

Key vaults

Default Directory (jobsmacoroprotonmail.onmicrosoft.com)

+ Create [Manage deleted vaults](#) [Manage view](#) [Refresh](#) [Export to CSV](#) [Open query](#) | [Assign tags](#) ... [Group by none](#)

Filter for any field...

Subscription equals all

Resource Group equals all

Location equals all

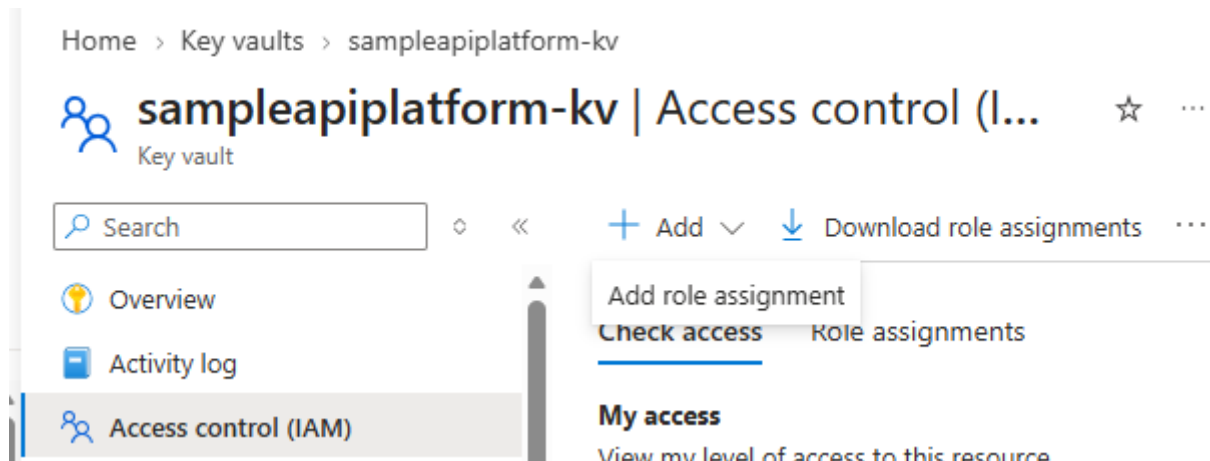
+ Add filter

☐	Name ↑	Type	Resource Group	Location	Subscription	Tags
☐	 sampleapiplatform-kv	...	Key vault	sampleApiPlatform_...	France Central	Azure subscription 1

Generate Secrets In The Key Vault

Before creating a secret we need to grant the user that is creating the secret permission to do that:

Go to the sampleapiplatform-kv Key Vault > Access Control (IAM) > Click on “Add role assignment”:



A list will be presented. Search for Key Vault Administrator (this role can create, delete, update secrets, manage keys and policies) > select it and click on the Next button:

Home > Key vaults > sampleapiplatform-kv | Access control (IAM)

Add role assignment

Role Members Conditions Review + assign

A role definition is a collection of permissions. You can use the built-in roles or you can create your own custom roles. [Learn more](#)

Copilot can help pick a role

Job function roles Privileged administrator roles

Grant access to Azure resources based on job function, such as the ability to create virtual machines.

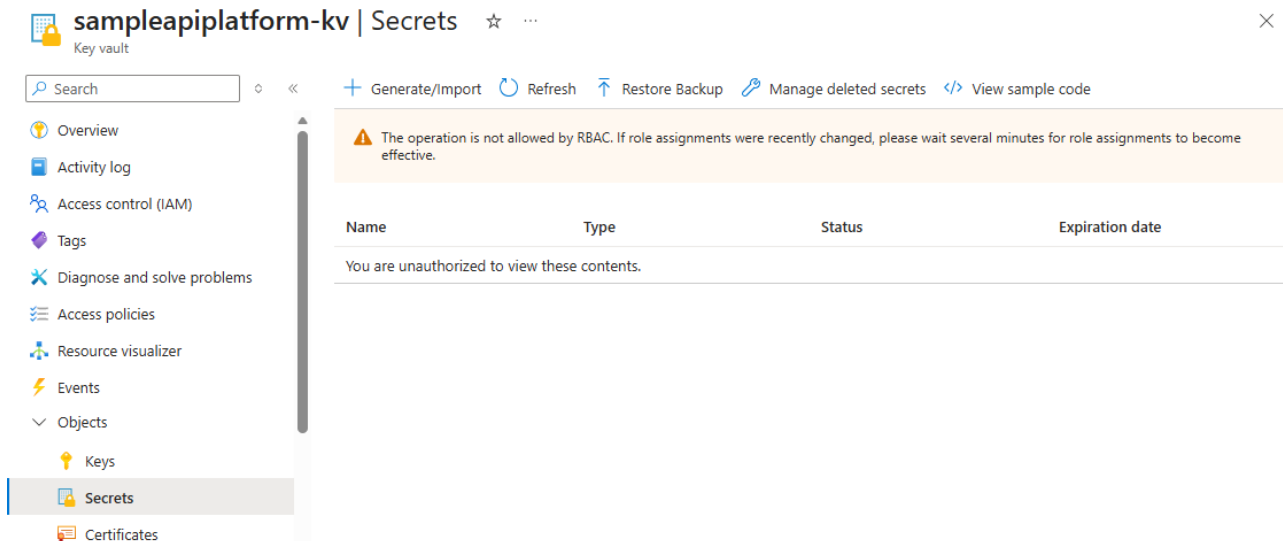
key vault Type: All Category: All

Name ↑↓	Description ↑↓	Type ↑↓	Category ↑↓	Details
Key Vault Administrator	Perform all data plane operations on a key vault and all objects in it, inclu...	BuiltInRole	Security	View
Key Vault Certificate User	Read certificate contents. Only works for key vaults that use the 'Azure ro...	BuiltInRole	None	View
Key Vault Certificates Officer	Perform any action on the certificates of a key vault, except manage per...	BuiltInRole	Security	View
Key Vault Contributor	Lets you manage key vaults, but not access to them.	BuiltInRole	Security	View
Key Vault Crypto Officer	Perform any action on the keys of a key vault, except manage permission...	BuiltInRole	Security	View

Review + assign Previous Next

An screen will be presented. Click on “Select members” and choose your account and click on Select and finally click on the “Review + assign” button.

Go to the sampleapiplatform-kv Key Vault > Objects > Secrets > click on the Generate / Import button :



Then we are presented with the Create a Secret screen:

Upload options = Manual

Name = sampleapiplatform-token-client1

Secret Value = 4f8b2e1c-9a2d-4c3f-8a1e-2c9d7f1e3b4a (a guid for instance. The value will be secret and only the clients get this value)

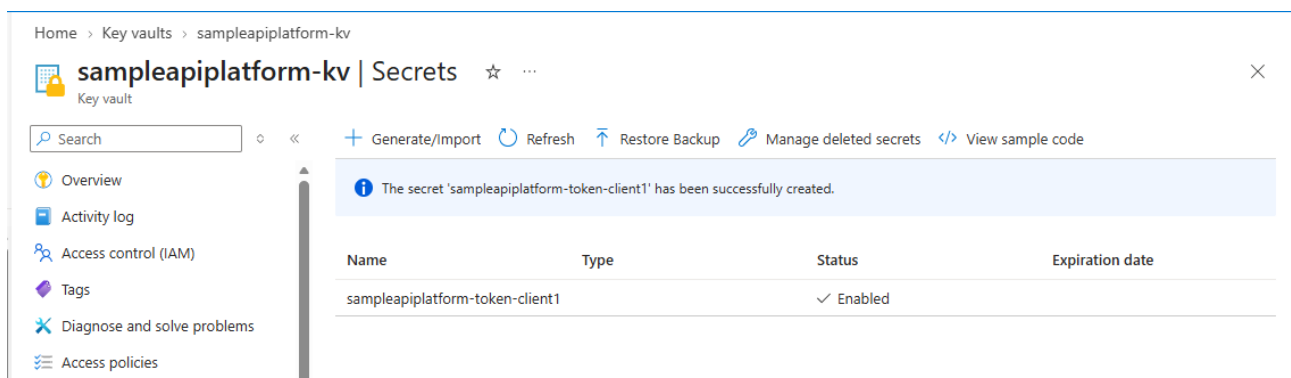
Content Type (optional) = empty (this is for metadata)

Set activation date = checkbox unchecked

Set expiration date = checkbox unchecked

Enabled = Yes (it needs to be active)

Click on the Create Button and the secret will be created:



Connect the SampleAppApi with the Key Vault

Activate the system assigned identity for the sampleappapi container app: Go to Container Apps > sampleappapi (container app) > Security > Identity > System Assigned = On and click on the Save button:

Home > Container Apps > sampleappapi

sampleappapi | Identity

Container App

Search

- Overview
- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Resource visualizer
- Application
- Settings
- Networking
- Security
 - Secrets
 - Authentication
 - Identity**

System assigned

User assigned

A system assigned managed identity is restricted to one per resource and is tied to the managed identity by using Azure role-based access control (Azure RBAC). The n you don't have to store any credentials in code. [Learn more about Managed identiti](#)

Save Discard Refresh Got feedback?

Status ⓘ

☒ On

Go to the sampleapiplatform-kv > Access Control (IAM) > Add Role Assignment (similar to what we have been doing before to grant permission to a user to create a secret) > Look for the “Key Vault Secrets User”

Home > Key vaults > sampleapiplatform-kv | Access control (IAM)

Add role assignment

Role Members Conditions Review + assign

A role definition is a collection of permissions. You can use the built-in roles or you can create your own custom roles. [Learn more](#)

[Copilot can help pick a role](#)

Job function roles

Privileged administrator roles

Grant access to Azure resources based on job function, such as the ability to create virtual machines.

Key Vault Secrets User Type: All Category: All

Name ↑↓	Description ↑↓	Type ↑↓	Category ↑↓	Details
Key Vault Certificate User	Read certificate contents. Only works for key vaults that use the 'Azure ro...	BuiltInRole	None	View
Key Vault Data Access Admi...	Manage access to Azure Key Vault by adding or removing role assignmen...	BuiltInRole	None	View
Key Vault Secrets User	Read secret contents. Only works for key vaults that use the 'Azure role-b...	BuiltInRole	Security	View

Showing 1 - 3 of 3 results.

Review + assign Previous Next

Feedback

And click on the “Next” button. Then on Assign Access to select Managed Identity and then click on “+ Select members” and choose the container app and then the sampleappapi and click on “Select”.

Home > Key vaults > sampleapiplatform-kv | Access control (IAM)

Add role assignment

Role: **Members** | Conditions | Review + assign

Selected role: Key Vault Secrets User

Assign access to: ☐ User, group, or service principal ☒ Managed identity

Members: + Select members

Name	Object ID
No members selected	

Description: Optional

Review + assign Previous Next

Select managed identities

Some results might be hidden due to your ABAC condition.

Subscription *: Azure subscription 1

Managed identity: Container App (1)

Search by name

Selected members:

- sampleappapi /subscriptions/c13b401c-dde4-4298-ad6c-3ef836f847d5/resourceGroups/... Remove

Select Close Feedback

Then on the “Review + assign” button:

Home > Key vaults > sampleapiplatform-kv | Access control (IAM)

Add role assignment

Role: **Members** | Conditions | Review + assign

Selected role: Key Vault Secrets User

Assign access to: ☐ User, group, or service principal ☒ Managed identity

Members: + Select members

Name	Object ID	Type
sampleappapi	a36e4b77-d461-4de8-8b56-93d918...	Container App ⓘ

Description: Optional

Review + assign Previous Next Feedback

Set the secret in the SampleAppApi Container App

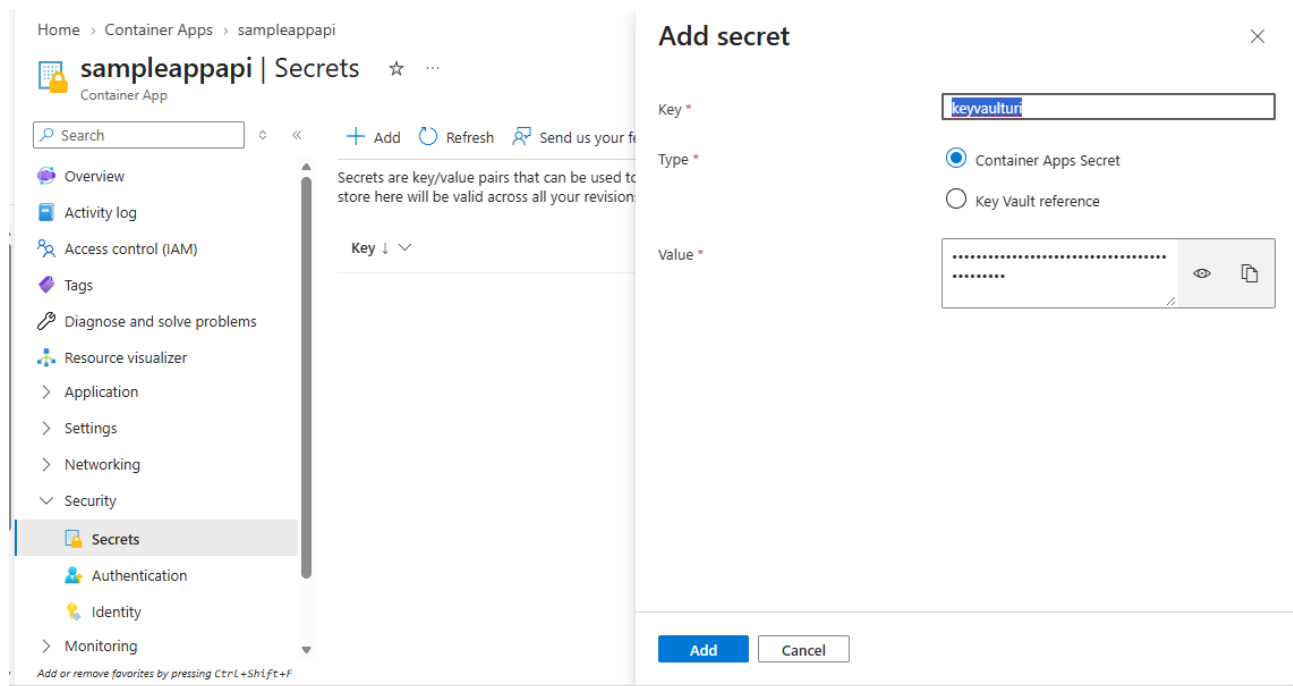
ContainerApps > sampleappapi > security > secrets > click on Add and fill out the the form with the following values:

Key = keyvaulturi

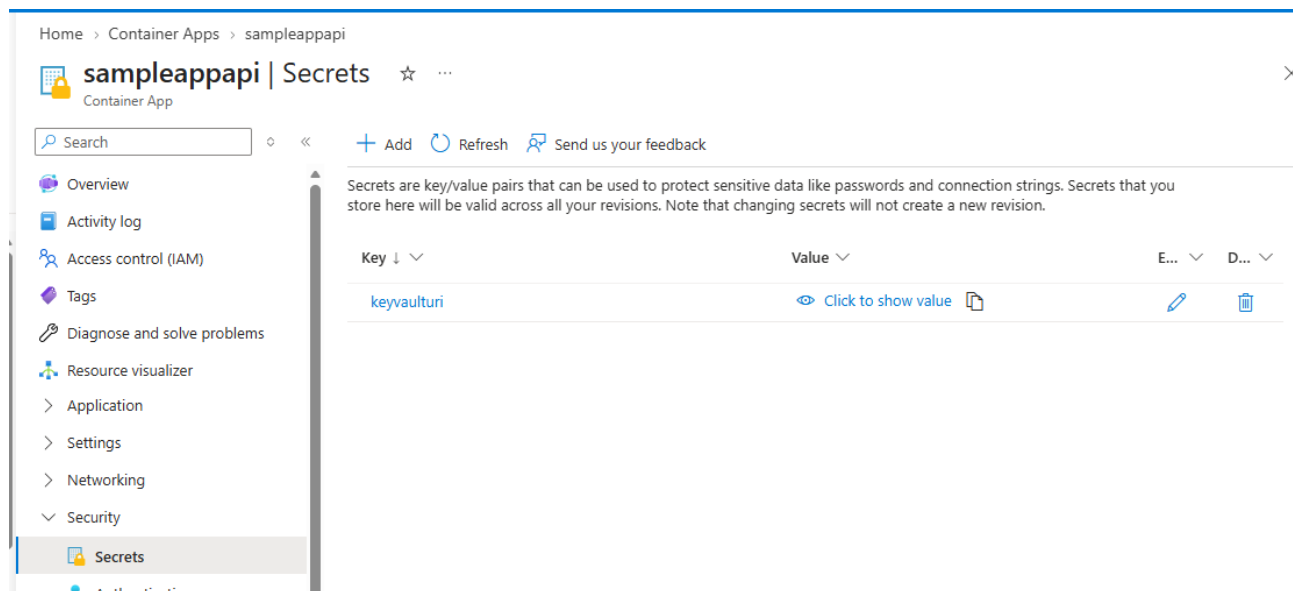
Type = Container App

Value = <https://sampleapiplatform-kv.vault.azure.net/> (this is the url of our key Vault that can be found in the Key Vault > Overview)

Click on the Add button:



If everything is successful then we should see the created secret:



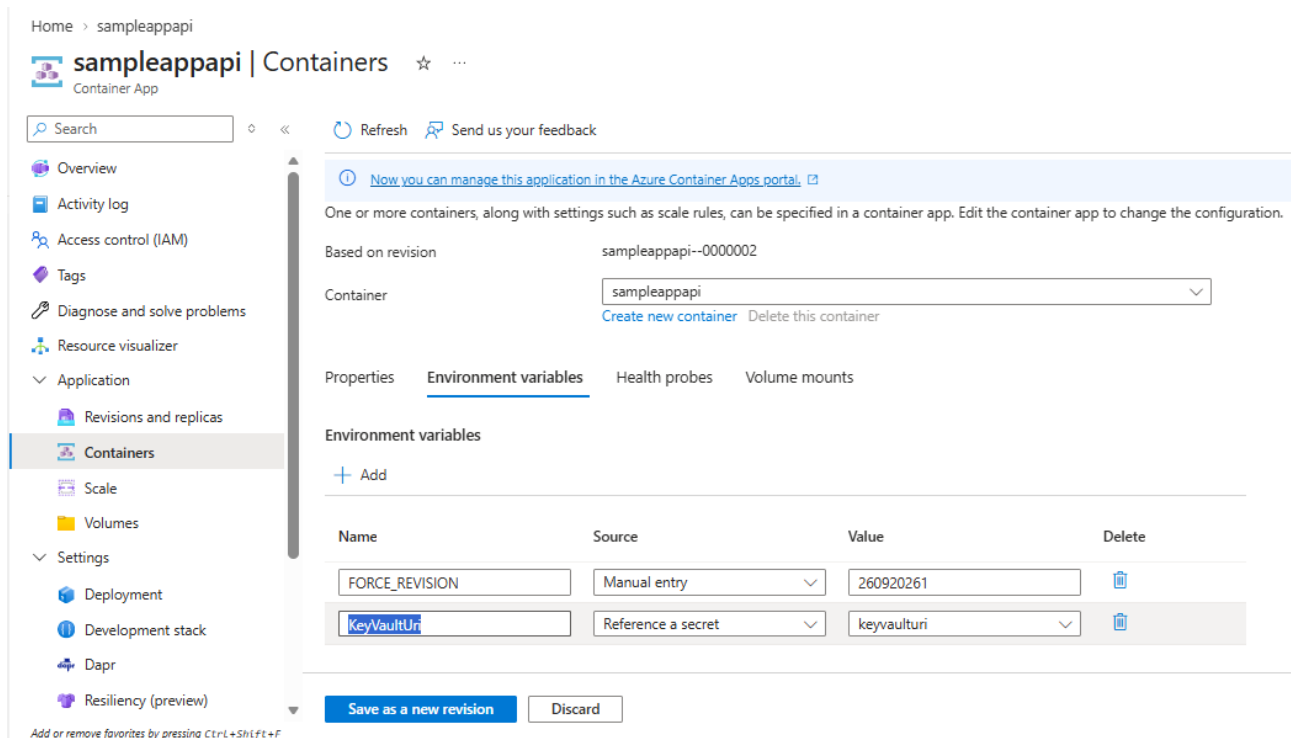
Create an environment variable

Go to the sampleappapi container app > Application > Containers > Environment Variables Tab.
Create the following environment variable:

Name=KeyVaultUri

Source=Reference a secret

value= keyvaulturi (this value will be available automatically in the drop down menu)



Program.cs: integrate the key vault

Install the following NuGet packages:

```
dotnet add package Azure.Extensions.AspNetCore.Configuration.Secrets
```

```
dotnet remove package Azure.Core
```

```
dotnet remove package Azure.Identity
```

```
dotnet add package Azure.Identity --version 1.11.0
```

```
dotnet add package Azure.Extensions.AspNetCore.Configuration.Secrets --version 1.2.2
```

Add the lines in bold to Program.cs in the SampleAppApi code:

```
using Azure.Identity;  
using Azure.Extensions.AspNetCore.Configuration.Secrets;  
using SampleAppApi.Extensions;  
using SampleAppApi.Interfaces.ExternalServices;  
using SampleApi.Options;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// -----
```

```
// 1 KeyVaultUri loaded from the Container App Secret
```

```
// -----
```

```

var keyVaultUri = builder.Configuration["KeyVaultUri"];

if (string.IsNullOrEmpty(keyVaultUri))
{
    throw new InvalidOperationException("KeyVaultUri configuration value is missing.");
}
// -----
// [2] Key Vault integrate
// -----
builder.Configuration.AddAzureKeyVault(
    new Uri(keyVaultUri),
    new DefaultAzureCredential());

builder.Services.AddScoped<IDataAccessClient, DataAccessClientDapr>();

//Swagger
builder.Services.AddSwaggerDocumentation();

// Controllers
builder.Services.AddControllers();

// -----
3 [3] ServiceTokenOptions load (Tokens from the Key Vault)
// -----
builder.Services.Configure<ServiceTokenOptions>(options =>
{
    var token1 = builder.Configuration["sampleapiplatform-token-client1"];

    if (string.IsNullOrEmpty(token1))
    {
        throw new InvalidOperationException("Token 'sampleapiplatform-token-client1' is
missing from Key Vault.");
    }

    options.ValidTokens.Add(token1);
});

// -----
// [4] Token-Middleware registrieren
// -----
builder.Services.AddSingleton<TokenValidationMiddleware>();

// Authorization
builder.Services.AddAuthorization();
// Dapr Client aktivieren (GENAU HIER)
builder.Services.AddDaprClient();
builder.Services.AddHttpClient("dapr", c =>
{
    c.BaseAddress = new Uri("http://localhost:3500/");
});

```

```

var app = builder.Build();
// this is not necessary: GlobalExceptionHandler
app.UseMiddleware<GlobalExceptionHandler>();

if (app.Environment.IsDevelopment() || app.Environment.IsStaging())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

//Here we check if the token is valid or not. It will check if the request header has a valid X-
//API-SECRET. If not will return 401
//put it before the authentication and Authorization
//blocks bots/scanners before they hit the auth logic
app.UseMiddleware<TokenValidationMiddleware>();

// ★ Authentication + Authorization middleware
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();

```

Create the following classes:

SampleApiApp > Options > ServiceTokenOptions.cs

```

namespace SampleApi.Options;

public class ServiceTokenOptions
{
    public List<string> ValidTokens { get; set; } = new();
}

```

And SampleApiApp > Middlewares > TokenValidationMiddleware.cs

```

using Microsoft.Extensions.Options;
using SampleApi.Options;

public class TokenValidationMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ServiceTokenOptions _options;

    public TokenValidationMiddleware(RequestDelegate next, IOptions<ServiceTokenOptions>
options)
    {

```

```

    _next = next;
    _options = options.Value;
}

public async Task InvokeAsync(HttpContext context)
{
    var token = context.Request.Headers["X-Service-Token"].FirstOrDefault();

    if (string.IsNullOrEmpty(token) || !_options.ValidTokens.Contains(token))
    {
        context.Response.StatusCode = StatusCodes.Status401Unauthorized;
        await context.Response.WriteAsync("Invalid or missing service token.");
        return;
    }

    await _next(context);
}
}

```

Deploy the new code and make sure that the pipeline finishes successfully.

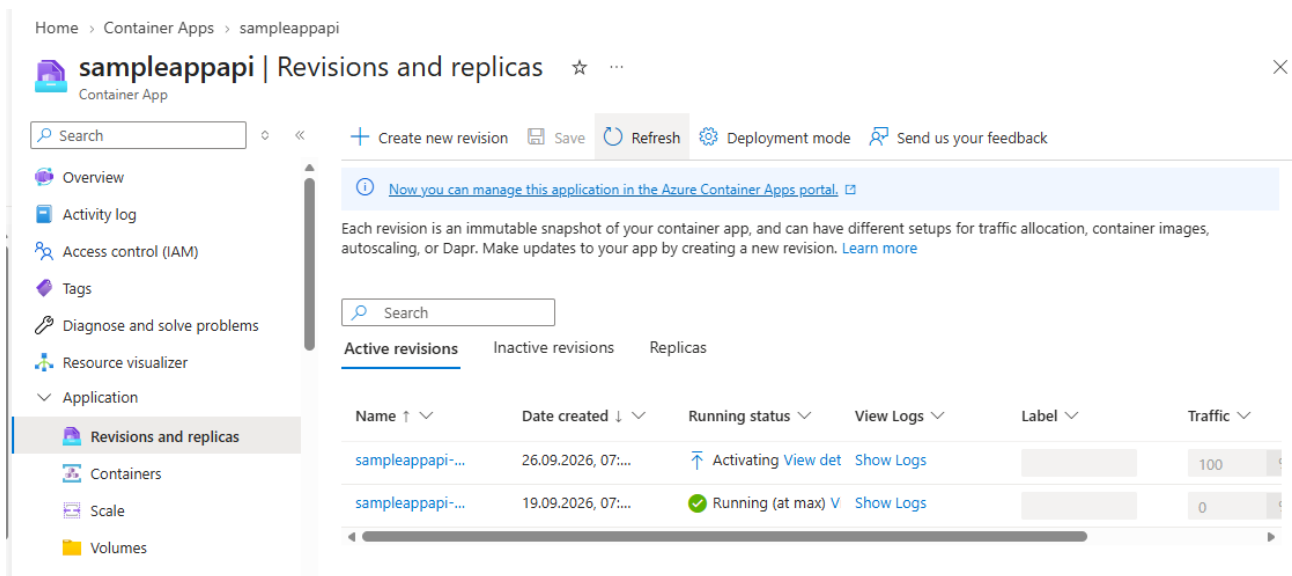
Restart the sampleappapi Container App

This is a necessary because container apps only load secrets when they start. We can do it from the shell:

```

az containerapp update \
--name sampleappapi \
--resource-group SampleAppApi_rg \
--set-env FORCE_REVISION=26092026

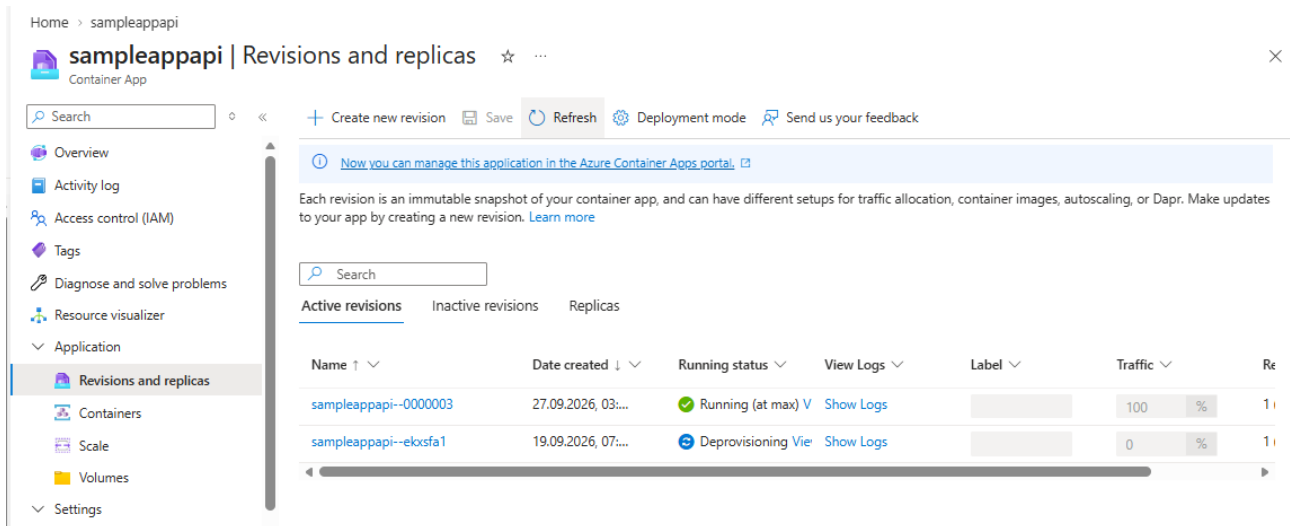
```



The screenshot shows the Azure Portal interface for a container app named 'sampleappapi'. The left sidebar contains navigation links: Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Resource visualizer, Application, Revisions and replicas (selected), Containers, Scale, and Volumes. The main content area is titled 'sampleappapi | Revisions and replicas' and includes a search bar and action buttons: '+ Create new revision', 'Save', 'Refresh', 'Deployment mode', and 'Send us your feedback'. A blue banner states: 'Now you can manage this application in the Azure Container Apps portal.' Below this, a text block explains: 'Each revision is an immutable snapshot of your container app, and can have different setups for traffic allocation, container images, autoscaling, or Dapr. Make updates to your app by creating a new revision. [Learn more](#)'. A table titled 'Active revisions' shows two revisions:

Name	Date created	Running status	View Logs	Label	Traffic
sampleappapi-...	26.09.2026, 07:...	Activating	View det Show Logs		100
sampleappapi-...	19.09.2026, 07:...	Running (at max)	Show Logs		0

So check that the new replica activates successfully



Check if the application is working

Sampleappapi Container App > overview > copy the url and add api/movies:

```
curl https://sampleappapi.redflower-2fd6d7ca.francecentral.azurecontainerapps.io/api/movies -H
"X-API-SECRET: 4f8b2e1c-9a2d-4c3f-8a1e-2c9d7f1e3b4a"
```

into the command line.

Check the logs and correct the errors.

If we correct an error and we change the code we should force a new replica to be generated. So run the following command from the Azure shell, by changing everytime the vale:

```
az containerapp update \
  --name sampleappapi \
  --resource-group SampleAppApi_rg \
  --set-env FORCE_REVISION=202609260540
```

Troubleshooting Errors

For instance after typing the following url in the browser:

sampleappapi.redflower-2fd6d7ca.francecentral.azurecontainerapps.io/api/movies

I get the following error in the browser:

```
{"status":500,"title":"Internal Server Error","detail":"An unexpected error
occurred.", "traceId":"0HNOS8V3U6ILA:00000002"}
```

This message was generated by the line in Program.cs:

```
app.UseMiddleware<GlobalExceptionMiddleware>();
```

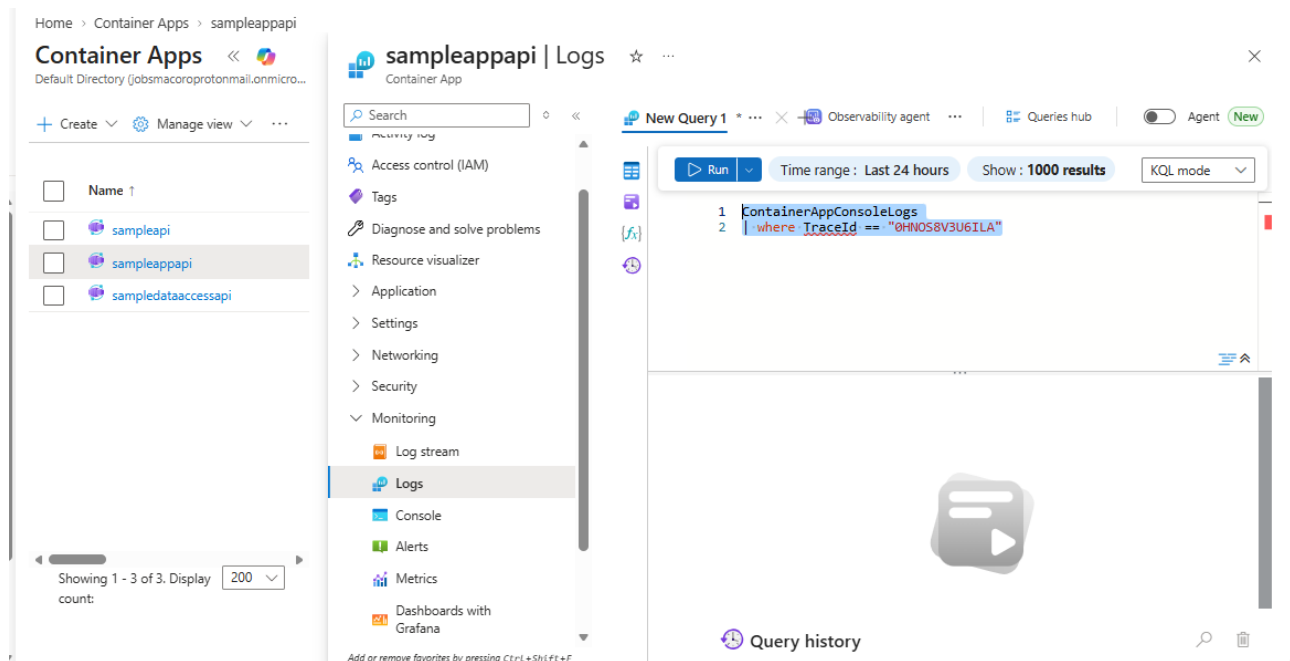
The class GlobalExceptionMiddleware genrates this message. If this line would not be here then the error would be exposed in the browser.

In this case the error is pointing to a certain traceId. We can try to find out more in the Logs.

Go to the sampleappapi Container App > Monitoring > Logs > Disable the Agent Toggle and use the KQL Mode (Kusto Query Language) and type the following query:

```
ContainerAppConsoleLogs  
| where TraceId == "0HNOS8V3U6ILA"
```

And click on the Run button



However this query did not give any result. So I uncommitted the line in Program.cs and now I was able to see the errors in the logs and the browser does not display any information only that a 500 error happened.

To Correct

The user Assigned identity is only needed with App1 App2 App3 wanted to access a resource. But with dapr in place in our situation is not necessary. Only for the database we need this identity but we can implement a System Assigned Identity instead.

I keep the User Assigned Identity to show to the employer

To access to the database: create a secret in the container sampledataaccessapi container with the following properties:

Key=moviesdbsecret

Type=Container Apps Secret

Value=Server=tcp:sampleapiserver.database.windows.net,1433;Database=free-sql-db-5542591;Authentication=Active Directory Default;Encrypt=True;

Notice that the value does not provide any password or user id because we are using the user defined identity.

Then create in the same container , sampledataaccessapi, an environment varibale :

Name=ConnectionStrings__MoviesDb

Source=Reference a secret

Value= moviesdbsecret

Properties Environment variables Health probes Volume mounts

Environment variables

[+ Add](#)

Name	Source	Value	Delete
ConnectionStrings__MoviesDb	Reference a secret	moviesdbsecret	

Created a new environment variable in sampledataaccessapi with the application id in order to connect to the database:

cliEntConnectionId:c93a55b4-29c9-439c-9fdf-f66f5e9eeeb9

```
TState state, CancellationToken cancellationToken)
2026-10-04T04:44:55.8146669Z stdout F      at Microsoft.EntityFrameworkCore.Storage.ExecutionStrategy.ExecuteAsync[TState,TResult](TState
state, Func`4 operation, Func`4 verifySucceeded, CancellationToken cancellationToken)
2026-10-04T04:44:55.8146765Z stdout F      at
Microsoft.EntityFrameworkCore.Query.Internal.SingleQueryingEnumerable`1.AsyncEnumerator.MoveNextAsync()
2026-10-04T04:44:55.8146801Z stdout F      ClientConnectionId:c93a55b4-29c9-439c-9fdf-f66f5e9eeeb9
2026-10-04T04:44:55.8146826Z stdout F      Error Number:-2,State:0,Class:11
2026-10-04T04:44:55.8147510Z stdout F fail: SampleDataAccessApi.Controllers.MoviesController[0]
2026-10-04T04:44:55.8147565Z stdout F      MovieController.GetAll Exception: Connection Timeout Expired. The timeout period elapsed during
the post-login phase. The connection could have timed out while waiting for server to complete the login process and respond; Or it could
have timed out while attempting to create multiple active connections. The duration spent while attempting to connect to this server was -
[Pre-Login] initialization=32; handshake=384; [Login] initialization=0; authentication=0; [Post-Login] complete=29161;
2026-10-04T04:46:15.87931 No logs since last 60 seconds
```

This clientConnectionId changes with each replica and needs to be updated. This value is only displayed when I issue an HTTP GET and not a POST.

Name = AZURE_CLIENT_ID

Source= Manual Entry

Value = c93a55b4-29c9-439c-9fdf-f66f5e9eeeb9

|

Home > Container Apps > sampledataaccessapi

sampledataaccessapi | Containers

Container App

Search

Refresh Send us your feedback

Create new container Delete this container

Properties **Environment variables** Health probes Volume mounts

Environment variables

+ Add

Name	Source	Value	Delete
ConnectionStrings__MoviesDb	Reference a secret	moviesdbsecret	
FORCE_REVISION	Manual entry	202610030324	
AZURE_CLIENT_ID	Manual entry	dd0c5b2a-7b64-487b-9280-32fbd4718a55	

Register Postman in EntraID

This is the part where we created a secret

Test Postman / curl

This has been covered with curl

Issues with Azure

Complex environement: until things wotk it takes long time to set up

Environment difficult to reproduce in Dev: secrets, dapr, tokens

Long process to debug: correct code, push into repo, run pipeline, update the container (cached)

HTTP Codes:

401= Un Authorized

